

**GHANA COMMUNICATION TECHNOLOGY
UNIVERSITY (GCTU)**

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING



**DEVELOPMENT AND IMPLEMENTATION OF A
ROBOTIC APPLICATION FOR HUMAN
INTENTION RECOGNITION USING MOTION AND
HAND GESTURE**

*A Project Work Submitted in Partial Fulfillment of the Requirements for the
Award of the Degree of*

BACHELOR OF SCIENCE IN COMPUTER ENGINEERING

BY:

ELEANA OSEI OWUSU (Index No.: 4121230024)

JOEL NII ADJETEY AHULU (Index No.: 4121230020)

SUPERVISOR:

MR. MICHEAL XENYA

SEPTEMBER, 2026

DECLARATION

Candidate's Declaration

This project is presented as part of the requirements for the award of the Degree of **Bachelor of Science in Computer Engineering** by Ghana Communication Technology University. We hereby declare that this project report is entirely the result of our own original research, hardware implementation, and empirical inquiries. We affirm that this project work has not been presented, in whole or in part, for the award of any other diploma or degree in this university or elsewhere. All sources of scholarly information, technical libraries, and literature have been duly cited and acknowledged.

Eleana Osei Owusu

(Index No.: 4121230024)

Signature: _____

Date: _____

Joel Nii Adjetey Ahulu

(Index No.: 4121230020)

Signature: _____

Date: _____

Supervisor's Certification

I hereby certify that this project work was carried out under my direct supervision in the Department of Computer Engineering, Faculty of Engineering, Ghana

Communication Technology University, in partial fulfillment of the requirements for the award of the Degree of Bachelor of Science in Computer Engineering.

Mr. Micheal Xenya

(Project Supervisor)

Signature: _____

Date: _____

Head of Department's Endorsement

I hereby confirm that this project work has been formally examined and accepted in satisfaction of the project requirements for the Department of Computer Engineering.

Dr. Samuel Danso

(Head of Department)

Department of Computer Engineering

Signature: _____

Date: _____

ABSTRACT

Deploying intuitive touchless human-robot interaction on resource-constrained mobile robots remains severely challenged by heavy GPU compute demands, network latencies inherent in cloud offloading, and the lack of physical spatial grounding in dynamic shared workspaces. This project presents the design, physical integration, and empirical evaluation of an autonomous mobile robot cognitive framework capable of real-time human intention recognition using skeletal motion and hand gestures executed entirely on embedded edge hardware. The physical testbed integrates an 8GB Raspberry Pi 5 single-board computer, a Yahboom micro-ROS ESP32-S3 motor controller, a 2D Time-of-Flight LiDAR, and a 2-DOF pan-tilt camera gimbal. The vision pipeline throttles camera intake to 20 Hz, enforces an empirical spatial acceptance zone to reject background bystanders, extracts 21 hand landmarks, and maps them into a 19-dimensional position- and scale-invariant geometric feature vector classified by a Multilayer Perceptron (MLP). A Long Short-Term Memory (LSTM) network anticipates human walking trajectories over an 8-frame horizon, while a supervisory finite state machine arbitrates motion commands via a ROS 2 priority multiplexer alongside 2D metric SLAM and Nav2 autonomous waypoint navigation. Rigorous physical trials across 180 experimental runs achieve a 99.38% offline test accuracy and a 96.67% physical operational accuracy across varying illumination and operator distances up to 2.5 meters. The total measured vision-to-actuation latency of 132.0 ms satisfies the 150 ms real-time latency deadline and complies with ISO 15066:2016 collaborative robot safety standards by executing immediate reactive halt preemption at a 0.36 meter LiDAR safety boundary.

Keywords: Human-Robot Interaction (HRI), Autonomous Mobile Robots, Edge Computing, Hand Gesture Recognition, ROS 2 Humble, Geometric Feature Invariance, Collaborative Safety (ISO 15066).

DEDICATION

This engineering project is wholeheartedly dedicated to our beloved parents and families, whose persistent prayers, profound sacrifices, and unwavering moral and financial support provided the bedrock upon which our academic aspirations were realized. Their boundless encouragement inspired us to persevere through demanding nights of laboratory experimentation, code debugging, and hardware fabrication.

We also dedicate this work to the prospective cohorts of computer engineering and robotics students at Ghana Communication Technology University, with the sincere hope that this open-source architecture and physical testbed serve as a practical foundation for groundbreaking advancements in autonomous systems, intelligent machines, and edge-native collaborative robotics across the African continent.

ACKNOWLEDGEMENTS

First and foremost, we give all glory, honor, and praise to the Almighty God for granting us divine wisdom, good health, endurance, and perseverance throughout the duration of this undergraduate engineering degree program and the successful completion of this project.

We register our profound gratitude and esteem to our project supervisor, **Mr. Micheal Xenya**, for his exceptional technical mentorship, critical reviews, and unwavering guidance. His meticulous attention to rigorous engineering design, standard-compliant methodology, and persistent encouragement challenged us to surpass surface-level simulations and realize a physically validated autonomous mobile robot.

We extend our sincere appreciation to the Head of Department, faculty members, laboratory technicians, and teaching assistants in the Department of Computer Engineering, Faculty of Engineering at Ghana Communication Technology University. Their academic instruction, provision of laboratory infrastructure, and administrative support enriched our academic experience.

Our heartfelt thanks go to the student volunteers and colleagues who patiently dedicated their time to participate in the 180 physical human interaction trials and gesture recognition experiments, providing the vital empirical dataset for our quantitative evaluations. Finally, we express our endless gratitude to our families and friends for their constant emotional support, patience, and encouragement throughout this journey.

Contents

DECLARATION	ii
ABSTRACT	iv
DEDICATION	v
ACKNOWLEDGEMENTS	vi
TABLE OF CONTENTS	xi
LIST OF TABLES	xiii
LIST OF FIGURES	xvi
LIST OF ABBREVIATIONS AND ACRONYMS	xvii
1 Introduction	1
1.1 Background of the Study	1
1.2 Problem Statement	2
1.3 Objectives of the Study	3
1.3.1 General Objective	3
1.3.2 Specific Objectives	3

1.3.3	Engineering Research Questions	4
1.4	Scope of the Study	5
1.5	Significance of the Study	6
1.6	Organization of the Study	6
2	Literature Review	8
2.0	Overview	8
2.1	Theoretical Review	8
2.2	Review of Related Works	9
2.2.1	Review One: Real-Time Intention Prediction via Upper-Limb Kinematics	9
2.2.2	Review Two: 3D Skeletal Tracking and Adaptive Gesture Classification	11
2.2.3	Review Three: Intention-Aware Navigation and Costmap Integration	13
2.2.4	Synthesis of Reviewed Literature	15
2.3	Comparative Summary of Additional Reviewed Literature	17
3	System Design and Methodology	19
3.1	Introduction	19
3.2	System Requirements	20
3.2.1	Hardware Requirements	20
3.2.2	Software Requirements	28
3.3	System Architecture Overview	30
3.3.1	Hardware Platform Specification	32
3.3.2	Software Framework and Deployment Strategy	33

3.3.3	Hardware Resource Planning and Memory Constraints . .	36
3.4	Data Acquisition and Dataset Preparation	36
3.4.1	Recording Conditions and Class Balancing	37
3.4.2	Preprocessing and Train-Test Splitting	37
3.4.3	Dataset Statistics	38
3.5	Perception and Intention Recognition Pipeline	39
3.5.1	Spatial Zone Person Detection	39
3.5.2	Vision-Based Hand Tracking	41
3.5.3	Geometric Feature Engineering	41
3.5.4	Gesture Classification Model	44
3.5.5	Human Movement Prediction (LSTM Path Predictor) . .	45
3.6	Centralized Decision-Making and Control	47
3.6.1	The Brain Node Architecture	47
3.6.2	Temporal Buffering and Locking Logic	48
3.6.3	Reactive Person Tracking (Visual Servoing)	49
3.7	ROS2 Middleware Integration and Topic Interfaces	50
3.7.1	ROS2 Domain Isolation	51
3.7.2	Sensor Data Streaming	51
3.7.3	Cognitive and Control Topologies	51
3.8	Mapping and Autonomous Navigation	52
3.8.1	SLAM Toolbox Configuration	52
3.8.2	Navigation2 (Nav2) Integration	54
3.9	ROS2 Node Graph Summary	54
3.10	Testing Instrumentation and Data Logging	55

4	Testing, Results and Analysis	56
4.1	Overview	56
4.2	Hardware and Sensor Testing	56
4.2.1	Visual Perception and Camera Testing	57
4.2.2	Planar LiDAR Sensing Testing	57
4.2.3	Embedded Edge Compute Node (Raspberry Pi 5) Testing	58
4.2.4	Microcontroller Actuation and Communication Bridge Test- ing	58
4.3	Gesture Recognition Testing	59
4.3.1	Test Procedure	59
4.3.2	Gesture Recognition Results	59
4.4	Movement Recognition Testing	61
4.4.1	Test Procedure	61
4.4.2	Movement Recognition Results	61
4.5	Robot Response Testing	62
4.5.1	Gesture-Based Robot Control	62
4.6	Mapping and Navigation Testing	63
4.6.1	Environment Mapping and Obstacle Avoidance	63
4.6.2	Empirical Single-Goal Autonomous Navigation Benchmark	64
4.6.3	Empirical Multi-Waypoint Autonomous Facility Patrol Bench- mark	66
4.7	Performance Evaluation and Benchmarking	68
4.7.1	Performance Metrics	68
4.7.2	Quantitative End-to-End Latency Budget	71

4.7.3	Benchmarking Analysis	72
4.8	Overall System Evaluation	73
4.9	Summary	74
5	Conclusion and Recommendation	76
5.1	Summary of the Study	76
5.2	Conclusion	77
5.3	Limitations of the Study	79
5.4	Recommendations for Future Work	81
	REFERENCES	83

List of Tables

2.1	Comparative Summary of Reviewed Literature in Gesture, Intention, and Robot Navigation	17
3.1	Hardware Components, Technical Specifications, and Functional Roles	27
3.2	Software Components, Deployment Targets, and System Roles . .	29
3.3	Per-Class Sample Distribution	38
3.4	Train / Validation / Test Split	38
3.5	Engineered Geometric Features (19 total)	43
3.6	Deployed ROS2 Nodes, Topics, and Services	55
4.1	Gesture Recognition Physical Trial Results (30 trials per gesture, 180 total)	59
4.2	Movement Recognition Test Results	62
4.3	Gesture-Based Robot Control Test Results	63
4.4	Obstacle Avoidance and Mapping Test Results	64
4.5	Empirical Multi-Waypoint Autonomous Patrol Performance Metrics	67
4.6	Per-Class Precision and Recall of the MLP Gesture Classifier . . .	69
4.7	Measured Real-Time System Throughput Across Active Nodes . .	70
4.8	Measured End-to-End Latency Budget Across the Processing Pipeline	71

4.9	Benchmarking Against Reviewed Gesture Recognition Systems . .	72
-----	---	----

List of Figures

3.1	Fully Assembled Yahboom MicroROS-Pi5 Autonomous Mobile Robot Platform	21
3.2	Raspberry Pi 5 Single-Board Computer (8GB LPDDR4X Edge Compute Node)	22
3.3	Yahboom MicroROS Robot Expansion Board (ESP32-S3 Co-Processor)	23
3.4	MS200 Time-of-Flight (ToF) 2D Planar LiDAR Sensor	24
3.5	2-DOF Active Pan-Tilt Camera Gimbal Mechanism	25
3.6	Internal Chassis Hardware Integration and Mechatronic Wiring Harness (Uncased View)	26
3.7	End-to-End System Architecture and Cognitive Signal Flow . . .	32
3.8	Hardware System Interconnection and Power Topology	33
3.9	Containerized Multi-Node Deployment Architecture (Docker Host Mode)	35
3.10	Empirical Spatial Receptive Zone Gating and Bystander Suppression Flow	40
3.11	Real-Time MediaPipe Feature Engineering and Invariant Neural Classifier Pipeline	45
3.12	LSTM Path Predictor Training and Validation Loss Convergence	46
3.13	LSTM Path Prediction — History vs. True vs. Predicted (Sample Sequences)	46

3.14	OMG UML 2.5 Brain State Machine and Safety Preemption Architecture	49
3.15	Comparison of Physical SLAM Occupancy Grid Maps During Platform Calibration: (a) Initial mapping attempt exhibiting the “hour-glass” rotational drift defect due to uncorrected IMU yaw; (b) Final clean metric map (third mapping attempt) achieved after enabling wheel odometry yaw fusion in the EKF.	54
4.1	Observed physical gesture classification accuracy across (a) discrete vocabulary classes and (b) experimental testing regimes (operational distance and ambient illumination) across all 180 physical trials.	60
4.2	Per-Class Classification Accuracy of the Trained MLP Gesture Model	61
4.3	Empirical physical robot navigation trajectory and kinematics decoded from wheel odometry telemetry during the 1.5 m autonomous navigation benchmark run. Subplots illustrate the 2D spatial path with heading quivers (left), chassis heading deviation over time demonstrating the rightward costmap repulsion maneuver (top-right), and forward linear velocity showing smooth acceleration, cruise (0.236 m/s), and terminal deceleration to goal arrival (bottom-right).	65
4.4	Empirical multi-waypoint autonomous patrol telemetry decoded from physical ROS 2 bag database (<code>patrol_20260913_215558</code>). Subplots illustrate: (a) 2D spatial trajectory across the calibrated Geometric L-Corridor topology ($P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$ and return) with continuous mission timestamp colormap and chassis heading quivers; (b) Cross-track error (MAE = 15.5 cm, RMSE = 22.2 cm) and chassis yaw orientation relative to the 12.0 cm goal tolerance boundary; and (c) Dynamic locomotion kinematics displaying linear velocity v_x (cruising at 0.20 m/s), angular yaw rate ω_z , and zero-velocity sensor dwell intervals.	67
4.5	Confusion Matrix for the MLP Gesture Classifier on the Test Dataset	69

4.6	Empirical end-to-end command turnaround latency distributions across (a) operational distance and (b) ambient lighting regimes recorded over 180 physical trials. Boxplots depict median and interquartile ranges with overlaid semi-transparent jittered scatter points, referencing the nominal 132.0 ms design budget and 150.0 ms ERQ 1 safety threshold.	72
-----	---	----

LIST OF ABBREVIATIONS AND ACRONYMS

Abbreviation	Full Definition / Expansion
ADE	Average Displacement Error
AI	Artificial Intelligence
AMCL	Adaptive Monte Carlo Localization
AMD	Advanced Micro Devices (<i>x86_64</i> Workstation Architecture)
ANOVA	Analysis of Variance
ARM	Advanced RISC Machine (Raspberry Pi 5 Architecture)
BOM	Bill of Materials
BPTT	Backpropagation Through Time
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
DC	Direct Current
DDS	Data Distribution Service (OMG Standard)
DOF	Degrees of Freedom
DWB	Dynamic Window Approach Controller
EKF	Extended Kalman Filter
ERQ	Engineering Research Question
ESP32-S3	Espressif Systems 32-bit Dual-Core Microcontroller
FDE	Final Displacement Error
FoV	Field of View
FPS	Frames Per Second
FSM	Finite State Machine
GCTU	Ghana Communication Technology University
GPIO	General Purpose Input/Output
GPU	Graphics Processing Unit
HMM	Hidden Markov Model

Abbreviation	Full Definition / Expansion
HRI	Human-Robot Interaction
I2C	Inter-Integrated Circuit Serial Bus
IBVS	Image-Based Visual Servoing
IMU	Inertial Measurement Unit
IQR	Interquartile Range
ISO	International Organization for Standardization
LiDAR	Light Detection and Ranging
LPDDR4X	Low-Power Double Data Rate 4X Synchronous DRAM
LSTM	Long Short-Term Memory Recurrent Neural Network
MAE	Mean Absolute Error
MCP	Metacarpophalangeal Joint
MLP	Multilayer Perceptron Classifier
MPU	Motion Processing Unit (MPU6050 6-Axis IMU)
MSE	Mean Squared Error
Nav2	Navigation 2 Autonomous Stack for ROS 2
NPU	Neural Processing Unit
OMG	Object Management Group
ONNX	Open Neural Network Exchange Format
OOM	Out of Memory
PIP	Proximal Interphalangeal Joint
PWM	Pulse Width Modulation
QoS	Quality of Service
RAM	Random Access Memory
REP	ROS Enhancement Proposal
RMSE	Root Mean Square Error
RMW	ROS Middleware Interface
ROS 2	Robot Operating System 2
SBC	Single-Board Computer (Raspberry Pi 5)
sEMG	Surface Electromyography
SLAM	Simultaneous Localization and Mapping
SVM	Support Vector Machine
TF2	Transform Library Version 2 (Spatial Frame Tree)
ToF	Time-of-Flight (MS200 Laser Scanning)
UART	Universal Asynchronous Receiver-Transmitter
UML	Unified Modeling Language
USB	Universal Serial Bus
YOLO	You Only Look Once (Real-Time Object Detector)

Chapter 1

Introduction

1.1 Background of the Study

Robotics engineering has transitioned rapidly from isolated industrial work-cells toward dynamic, human-centered environments where machines and humans share physical workspaces. In these collaborative settings, autonomous mobile robots are required to operate intelligently alongside human personnel rather than merely executing rigid, pre-programmed trajectories in segregated areas. This operational shift has established Human-Robot Interaction (HRI) as a foundational discipline in modern robotics, requiring machines to interpret human social cues and intentions with high operational reliability. For an autonomous mobile robot to function safely and intuitively in shared spaces, it must recognize, interpret, and respond to human behavior in a manner that is predictable, natural, and immediate.

A primary communication channel in human-to-human collaboration is the use of natural hand gestures and body movement cues. Although modern computer vision algorithms have made optical gesture recognition increasingly practical, conventional deep learning implementations typically rely on cloud infrastructure or power-intensive Graphics Processing Units (GPUs) [1]. This reliance introduces severe communication latencies and vulnerability to network interruptions, rendering remote processing impractical for time-critical robotic safety applications [2]. Furthermore, a mobile robotic system cannot operate effectively by merely classifying gestures in isolation; it must maintain active spatial awareness of its surrounding environment to execute navigational directives safely. This study addresses these interconnected challenges by engineering and physically de-

ploying a unified, vision-based intention recognition and autonomous navigation pipeline operating strictly on an embedded edge computing platform.

1.2 Problem Statement

Despite significant advancements in collaborative robotics, achieving seamless, responsive, and robust human-robot collaboration on low-cost, resource-constrained hardware remains an unresolved engineering bottleneck. Most industrial and commercial service robots still depend on rigid control modalities, such as physical teach pendants, capacitive touchscreens, or acoustic voice commands. These conventional interfaces frequently prove ineffective in noisy industrial manufacturing facilities or sterile healthcare environments where physical contact must be strictly avoided.

A critical analysis of state-of-the-art vision architectures reveals several practical barriers that impede reliable field deployment. Although laboratory prototypes frequently demonstrate high benchmark accuracy, their real-world reliability deteriorates significantly under unconstrained operational conditions. Specifically, existing vision-based intention recognition frameworks exhibit three critical technological limitations when transitioned to real-world embedded deployment:

1. **High-Power Hardware and Cloud Dependency:** Many contemporary vision-based gesture architectures rely heavily on high-end desktop GPUs or offboard cloud computing services for inference [1]. In mobile robotics, reliance on offboard processing introduces unpredictable network latency, consumes excessive power, and introduces single-point network failure risks that violate real-time safety thresholds [2].
2. **Spatial Coordinate Domain Shift Across Distances:** Standard machine learning classifiers frequently evaluate raw anatomical pixel coordinates extracted from image frames. Because absolute landmark coordinates vary drastically as an operator moves closer to or farther from the camera, conventional models suffer severe classification degradation under real-world operational distance variations.
3. **Ungrounded and Spatial-Blind Command Execution:** Existing gesture-controlled mobile robots predominantly execute motor commands blindly without contextual awareness of their physical surroundings. Translating

recognized gestures into motor velocities without continuous laser-based environment mapping and obstacle verification presents immediate collision hazards in populated indoor workspaces.

To overcome these engineering limitations, there is a distinct requirement for a lightweight, edge-deployable framework that tightly couples real-time intention recognition with spatial mapping within a unified, standardized robotic middleware. Such a framework must operate deterministically within embedded compute and thermal constraints while maintaining uncompromised collaborative safety. Furthermore, bridging the gap between high-level visual perception and low-level chassis actuation is essential to prevent latency bottlenecks that could lead to erratic or dangerous vehicular maneuvers.

1.3 Objectives of the Study

1.3.1 General Objective

The general objective of this research is to design, implement, and empirically evaluate an edge-deployed, real-time human intention recognition and navigation system that integrates scale-invariant hand gesture classification, temporal motion prediction, and spatial mapping into a unified ROS2 middleware pipeline on an autonomous mobile robot. This involves establishing a complete perception-to-action control loop that executes reliably on a physical mobile testbed without dependence on remote servers. In doing so, the study demonstrates the feasibility of deterministic human-robot collaboration under realistic indoor operational constraints.

1.3.2 Specific Objectives

Translating the broad engineering goal into a functioning cyber-physical system requires a structured multi-stage development methodology. Each phase addresses a specific perception, decision-making, or navigation challenge within the embedded compute envelope. To achieve this general objective, the following specific objectives were established:

1. To design and implement an edge-optimized computer vision pipeline uti-

lizing spatial zone filtering, anatomical hand landmark tracking, and a Multilayer Perceptron (MLP) classifier trained on nineteen engineered, scale-invariant geometric hand features.

2. To construct, balance, and validate a custom 6,000-sample gesture dataset captured directly through the onboard monocular camera to eliminate sensor domain shift across six distinct operational commands.
3. To develop a sequential motion prediction model utilizing a Long Short-Term Memory (LSTM) recurrent neural network to forecast short-horizon operator movement trajectories from continuous body pose landmarks.
4. To implement a centralized, deterministic decision-making node (**brain_node**) featuring temporal majority-vote confirmation, active subject-locking, and proportional visual servoing within a multi-node ROS2 Humble software architecture.
5. To configure and physically validate Simultaneous Localization and Mapping (SLAM) and autonomous navigation stacks (Nav2) on the mobile platform to support safe, spatially aware trajectory execution.
6. To quantitatively evaluate system performance through classification accuracy, confusion matrix analysis, component latency budgets, and real-time hardware throughput measurements on an embedded edge processor.

1.3.3 Engineering Research Questions

To provide rigorous scientific evaluation, the experimental investigation is guided by formal engineering inquiries that test core hypotheses regarding system performance. These inquiries systematically interrogate the mathematical boundaries of the feature representation, computational latency, and collaborative safety. In alignment with Faculty of Engineering standards, this study formulates four core Engineering Research Questions (ERQs) to evaluate the technical feasibility, mathematical invariance, multi-user robustness, and safety compliance of the deployed system:

1. **ERQ 1 (Edge Feasibility and Latency Budget):** Can a multi-stage visual perception pipeline—incorporating person detection, twenty-one-point hand landmark extraction, and geometric feature computation—execute natively on a low-cost, low-power edge processor (Raspberry Pi 5) without

GPU acceleration while maintaining an end-to-end processing cycle below the 150 ms human-interaction latency threshold?

2. **ERQ 2 (Mathematical Feature Invariance):** To what extent does converting raw anatomical landmark coordinates into scale- and distance-invariant geometric features (joint curl angles and palm-normalized Euclidean distances) eliminate spatial distortion and improve gesture classification accuracy across varying operator distances compared to unnormalized coordinate baselines?
3. **ERQ 3 (Multi-Person Workspace Disambiguation):** How effectively can a central geometric spatial-zone filter coupled with rolling majority-vote temporal buffering isolate and maintain an active control lock on a primary operator in a shared workspace, preventing unintended command execution from background bystanders?
4. **ERQ 4 (Intention-to-Action Coupling and Safety Verification):** Can discrete hand gesture commands and continuous kinematic motion trajectories be reliably integrated within ROS2 middleware to govern mobile robot velocity commands in full compliance with ISO 15066 collaborative robotic safety and reaction requirements?

1.4 Scope of the Study

The operational scope of this research is bounded to the design, software integration, and physical deployment of a gesture-directed autonomous mobile robot operating within indoor environments. The testing environment is restricted to structured indoor settings with stable floor surfaces and standard ambient fluorescent illumination to ensure predictable sensor characteristics. To maintain deterministic control and prevent command ambiguity, the visual interaction vocabulary is restricted to six predefined static hand gestures (STOP, GO, LEFT, RIGHT, BACK, FOLLOW) and five discrete body motion states (Approaching, Moving Away, Moving Left, Moving Right, Stationary).

Interaction is programmatically constrained to a single validated human operator at any given time, enforced through spatial-zone acceptance filtering and temporal lock scheduling. Furthermore, to address network vulnerability and communication latency constraints, the entire perception, intention interpretation, and motor control software stack operates strictly onboard the robot's local

processor without any reliance on remote cloud servers or offboard computing clusters. Real-time operation is defined as the capability of the onboard architecture to sustain an end-to-end sensing-to-actuation cycle sufficient to maintain responsive, jitter-free interaction at human operating speeds.

1.5 Significance of the Study

The engineering significance of this study lies in demonstrating that complex, multi-modal human intention recognition can be successfully decoupled from power-hungry workstation GPUs and external cloud infrastructure. By deploying lightweight, mathematically engineered feature extractors directly onto embedded edge hardware, this research provides a scalable, cost-effective blueprint for autonomous mobile robots operating in resource-constrained settings. This architectural approach dramatically reduces deployment expenses and operational power consumption while eliminating vulnerability to wireless network dropouts.

From a practical perspective, an autonomous mobile robot capable of interpreting touchless gestures and body motion addresses critical requirements across several industrial and civil domains. In acoustically noisy manufacturing environments, optical gesture recognition provides an intuitive, reliable control alternative where acoustic voice recognition routinely fails. In sterile medical facilities and cleanrooms, touchless optical interaction prevents physical contamination and preserves strict hygiene protocols by eliminating contact with shared touchscreens. Academically, this study contributes an open, modular ROS2 architecture that combines explicit gesture classification with predictive motion tracking and laser-based spatial mapping. Ultimately, this work supports safer, more intuitive human-robot collaboration by ensuring predictable machine responses that adhere to international safety standards for collaborative robotic workspaces [3].

1.6 Organization of the Study

This project report is systematically structured into five cohesive chapters in accordance with the university project guidelines. Chapter One introduces the research background, defines the specific engineering bottlenecks, formulates the general and specific objectives alongside the core Engineering Research Questions, and delineates the project’s scope and significance. Chapter Two presents

a comprehensive review of related literature, examining theoretical foundations in Human-Robot Interaction, analyzing prior works in gesture classification and intention-aware navigation, and identifying the precise technological gaps addressed by this research.

Chapter Three details the complete system design and methodology, encompassing physical hardware specifications, Docker-based containerized software deployment, geometric feature mathematics, LSTM path predictor training, and centralized state machine architecture. Chapter Four presents the empirical testing procedures, experimental results, classification confusion matrices, latency budgets, hardware throughput benchmarks, and comparative literature benchmarking. Finally, Chapter Five concludes the thesis by synthesizing the primary engineering findings, discussing physical implementation constraints, and proposing concrete recommendations for future industrial extensions.

Chapter 2

Literature Review

2.0 Overview

This chapter presents a critical review of relevant literature in human intention recognition, optical gesture analysis, and autonomous robot navigation in collaborative workspaces. Existing research establishes that autonomous mobile systems can recognize gestures, infer human motion trajectories, and adapt navigation behaviors to enhance interaction safety. However, a significant proportion of these systems operate under controlled laboratory conditions, depend on specialized high-power sensing hardware, or decouple intention recognition from mobile navigation. Consequently, this study builds directly upon existing theoretical and empirical frameworks by integrating vision-based human motion tracking and geometric hand gesture classification into an edge-deployed, mobile robotic platform operating in a mapped indoor environment.

2.1 Theoretical Review

This research is theoretically grounded in the principles of Human-Robot Interaction (HRI), human intention theory, geometric landmark kinematics, and spatial mobile robotics. Human intention theory posits that human goal-directed actions contain early predictive cues embedded within movement trajectories, forming the computational foundation for anticipatory robotic response systems. Within contemporary Human-Robot Interaction paradigms, collaborative mobile robots are required to move beyond purely reactive obstacle avoidance toward

proactive, context-aware interaction within human-shared operational domains.

The theoretical formulation of gesture and motion recognition in this study is based on the premise that human hand anatomical configurations can be modeled geometrically and topologically from single-frame landmark distributions. Static gesture classification relies on the principle that instantaneous spatial arrangements of hand joints provide sufficient discriminative features to distinguish discrete operational commands within a fixed vocabulary. Conversely, human movement trajectory prediction inherently requires temporal context, as instantaneous positions cannot reveal directional intent or velocity. This distinction necessitates a sequential modeling approach—specifically Long Short-Term Memory (LSTM) recurrent neural networks—to capture temporal dependencies across consecutive pose observations over a sliding temporal window.

Spatial mapping and localization theory provides the mathematical framework for continuous autonomous navigation between human interaction events. Simultaneous Localization and Mapping (SLAM) constructs an occupancy-grid representation of the physical environment, while probabilistic Monte Carlo algorithms estimate the robot’s coordinate pose relative to the global reference frame. Finally, multi-modal perceptual fusion theory demonstrates that combining explicit command channels (hand gestures) with implicit contextual cues (kinematic motion trajectories) mitigates command ambiguity and enhances operational robustness. Together, these theoretical frameworks provide the formal basis for developing a unified robotic system capable of recognizing, predicting, and responding to human intentions on physical hardware.

2.2 Review of Related Works

2.2.1 Review One: Real-Time Intention Prediction via Upper-Limb Kinematics

Research Topic and Authors

This investigation evaluates the peer-reviewed study entitled *Real-time Feasibility of a Human Intention Method Evaluated Through a Competitive Human–Robot Reaching Game*, published by Tsitos et al. (2022) in the *IEEE Transactions on Cognitive and Developmental Systems* [2]. The research was conducted by roboti-

cists at the Aristotle University of Thessaloniki, focusing on predictive biomechanics and rapid human intention inference. Their work provides a benchmark investigation into upper-limb skeletal kinematics and the latency constraints governing real-time human–robot collaboration.

Objectives

The primary objective of Tsitos et al. was to develop and evaluate an edge-feasible human intention prediction framework using a single RGB-D camera and the OpenPose skeletal tracking library. The authors aimed to integrate this predictive module into a standard ROS-based robotic control pipeline to govern arm trajectories in real time. Furthermore, the study investigated whether an industrial manipulator could react sufficiently early during competitive reaching tasks based on anticipatory kinematics rather than post-motion feedback.

Methodology

The authors implemented an experimental pipeline combining real-time human pose estimation with machine learning classification. An RGB-D camera captured upper-body human motion, from which OpenPose extracted 2D and 3D wrist joint coordinates during reaching movements. Raw spatial coordinates were digitally filtered to eliminate tracking jitter and segmented into distinct kinematic movement phases. Feature vectors were subsequently constructed from velocity and position profiles to train multiple classification models, including Support Vector Machines (SVM), Logistic Regression, Decision Trees, and Naive Bayes classifiers. The finalized predictive pipeline was interfaced directly into a ROS environment to control a 6-DOF Universal Robots UR3 manipulator during competitive reaching tasks.

Strengths

The investigation demonstrates that accurate intention prediction can be achieved using a streamlined, cost-effective setup comprising a single monocular depth sensor. By restricting the feature space to minimal wrist joint coordinates, the system sustained low-latency inference suitable for time-critical robotic interaction. The physical deployment on an industrial UR3 manipulator provided empirical validation on real hardware rather than simulated kinematics. Additionally, the

experimental identification of a strict human reaction time threshold emphasized the critical role of processing latency in collaborative robotics.

Limitations

A primary limitation of the study is that the evaluation was confined to a static tabletop reaching game without mobile base movement. The human operator and the robotic arm did not share a contiguous mobile floor space, preventing the evaluation of spatial navigation or dynamic obstacle avoidance. Furthermore, the system relied exclusively on implicit reaching motion, offering no explicit gesture command channel through which the human could issue deliberate supervisory directives.

Research Gap

While Tsitos et al. demonstrated that kinematic intention prediction can operate within strict latency bounds, their architecture is restricted to a stationary manipulator arm. The robot and human never navigate a shared physical floor environment, and the human possesses no mechanism to command the robot through deliberate, explicit gestures. This leaves a distinct engineering gap between fast, stationary intention inference and its deployment onto an autonomous mobile platform capable of accepting both explicit gesture commands and navigating physical spaces.

2.2.2 Review Two: 3D Skeletal Tracking and Adaptive Gesture Classification

Research Topic and Authors

This review examines the research paper entitled *3D Gesture Recognition and Adaptation for Human-Robot Interaction*, authored by Mahmud et al. (2022) and presented in leading international robotics conference proceedings [1]. The authors investigate spatial perception and dynamic gesture interpretation for mobile assistive robots using three-dimensional skeletal coordinate modeling. Their contribution focuses on bridging explicit human pointing directions with adaptive command parsing in interactive home and service robotics environments.

Objectives

The core objective of Mahmud et al. was to develop a three-dimensional hand gesture recognition and adaptation architecture capable of identifying both pointing and dynamic gestures in real time using depth sensing. The study sought to apply these classified gestures directly to robot navigation within human-robot interaction domains. Additionally, the authors aimed to establish a semi-supervised gesture adaptation mechanism that allowed the robotic system to incorporate novel user-defined gestures dynamically.

Methodology

The system employed a Microsoft Kinect depth sensor to capture 3D skeletal joint coordinates across the operator’s upper torso and hands. Pointing vectors were computed geometrically using 3D line equations projected onto the horizontal X–Z plane, while dynamic gestures were modeled from sequential joint trajectories over time. Spatial joint coordinates were normalized relative to a torso reference frame to maintain geometric consistency across different user physical builds. These feature representations were evaluated across three distinct machine learning classifiers: Hidden Markov Models (HMM), Support Vector Machines (SVM), and Convolutional Neural Networks (CNN). The navigation pipeline was evaluated within a simulated indoor environment using a dataset of 3,600 gesture instances collected from 24 participants across multiple age demographics.

Strengths

The utilization of three-dimensional skeletal coordinate data substantially enhanced tracking robustness against ambient lighting shifts and visual background clutter compared to conventional 2D vision techniques. The simultaneous integration of pointing vectors and dynamic gestures provided an expressive, natural interaction vocabulary for human operators. Moreover, the semi-supervised adaptation mechanism granted flexibility by allowing the system to update gesture vocabularies without requiring complete model retraining from scratch. The extensive demographic evaluation across multiple age brackets demonstrated practical insights into user adaptability across varying motor proficiencies.

Limitations

Despite these operational strengths, the architecture exhibits severe hardware and environmental constraints. The entire tracking pipeline depends strictly on the proprietary Microsoft Kinect hardware, restricting deployment to devices supporting heavy depth-sensor drivers. Furthermore, the navigation component was validated exclusively within a simulated software environment, leaving real-world sensor noise, wheel slip, and embedded latency unaddressed. The pointing estimation relied exclusively on right-hand joint configurations, which restricts natural interaction for left-handed operators or bi-manual interaction.

Research Gap

Mahmud et al. established a robust multi-modal gesture classification pipeline, but their navigation control was evaluated only in simulation and relied fundamentally on a high-power Kinect depth sensor. In contrast, non-visual approaches such as surface electromyography (sEMG) explored by Zafar et al. [4] eliminate visual occlusion issues, but impose the significant operational burden of wearable electrodes on the operator. This creates a compelling need for a completely device-free, monocular camera-based framework that performs gesture classification and spatial navigation entirely on edge hardware in real physical environments.

2.2.3 Review Three: Intention-Aware Navigation and Costmap Integration

Research Topic and Authors

This section critically examines the research paper entitled *Safe and Efficient Motion Planning for Material Transportation Robots Considering Intention Prediction of Obstacles*, published by Li et al. (2023) in the domain of autonomous industrial transport [5]. The authors address the persistent challenge of dynamic obstacle management in congested construction and logistics sites through intention-aware path planning. Their methodology investigates the fusion of visual deep learning and 2D LiDAR costmap layers within the ROS2 Navigation2 ecosystem.

Objectives

The primary objective of Li et al. was to formulate an intention-aware motion planning framework for autonomous transportation robots operating in congested industrial construction sites. The authors aimed to extend conventional static obstacle avoidance by predicting whether detected obstacles (specifically human workers) were likely to yield or move away from the robot’s projected trajectory. Furthermore, the study sought to integrate these intention predictions directly into ROS2 Navigation2 (Nav2) costmaps without modifying the underlying path planning algorithms.

Methodology

The authors developed a multi-sensor perception architecture combining 2D LiDAR scanning with camera-based convolutional object detection to classify site obstacles into distinct categories, including human workers, transport carts, machinery, and static materials. A predictive intention model based on Rectified Linear Unit (ReLU) activation functions estimated the probability of obstacle clearance based on semantic class, relative velocity, and Euclidean distance. When an obstacle was predicted to clear the path, the global costmap was dynamically updated to prevent unnecessary path replanning, while the local costmap maintained active collision safety. The complete navigation pipeline was implemented within the ROS2 Nav2 framework and evaluated using Isaac Sim on an autonomous Carter mobile platform across simulated construction environments.

Strengths

The central engineering strength of Li et al.’s framework is the dynamic modification of standard global costmaps based on predicted obstacle behavior, which significantly reduces unnecessary robot stops and inefficient path deviations. The multi-modal combination of LiDAR distance arrays and visual semantic detection provided rich spatial understanding in complex layouts. Furthermore, because the architecture interfaces directly with standard Nav2 costmap layers, it remains readily portable across different mobile base platforms without altering the core navigation solvers.

Limitations

A significant limitation is that the entire evaluation was conducted inside the NVIDIA Isaac Sim virtual simulation environment without physical robot deployment. As a consequence, physical motor actuation delays, real sensor dropouts, and embedded memory constraints were not verified on constrained edge processors. Furthermore, the intention recognition was entirely passive; the system possessed no explicit communication channel allowing human workers to deliberately direct, pause, or interact with the robot through gestures.

Research Gap

Li et al. demonstrated the architectural utility of modifying Nav2 costmaps using inferred human intentions, but their framework remains strictly passive and unverified on physical hardware. The robot infers whether a worker might clear the path, but the worker cannot issue an explicit command (such as an emergency stop or a directional waypoint) through natural visual gestures. This reveals a distinct research gap: combining an active, explicit gesture command channel with autonomous navigation on a physically deployed, edge-only mobile platform.

2.2.4 Synthesis of Reviewed Literature

A comprehensive analysis of the literature reveals a clear technological divide across current research. Systems that achieve exceptional intention prediction accuracy (such as Tsitos et al. [2]) operate predominantly on stationary robotic arms decoupled from mobile navigation. Conversely, frameworks that implement intention-aware mobile navigation (such as Li et al. [5] and Yu et al. [6]) rely solely on passive obstacle classification in virtual simulation environments, lacking explicit human-command interfaces. Furthermore, recent lightweight gesture architectures (such as Muhtadin et al. [7]) demonstrate edge-feasible MediaPipe classification on ROS2, but validate their models strictly on stationary UR5 industrial arms without mobile chassis or SLAM-based spatial mapping.

This study directly resolves this collective research gap by engineering a unified, physical mobile robotic platform that integrates active, scale-invariant hand gesture recognition, temporal motion prediction, and LiDAR SLAM mapping natively onto an embedded edge computing architecture. By decoupling perception

from external cloud infrastructure and high-power GPUs, the proposed architecture guarantees deterministic sensing-to-actuation latencies below international collaborative safety thresholds. Consequently, this research delivers an empirically validated, cost-effective framework that unifies explicit gesture commanding with continuous spatial navigation in dynamic human-shared environments.

2.3 Comparative Summary of Additional Reviewed Literature

Table 2.1: Comparative Summary of Reviewed Literature in Gesture, Intention, and Robot Navigation

Authors / Year	Research Topic	Methodology	Limitations	Research Gap	Proposed Improvement
Xinbo Yu et al. (2024) [6]	Real-Time Trajectory Planning and Obstacle Avoidance for Human-Robot Co-Transporting	Employs a physical slider-sensor mechanism and 2D LiDAR with a multiscale perception model to estimate human pulling motion and plan safe co-transporting trajectories.	Relies on physical contact via mechanical sliders; indirect intention inference easily perturbed by mechanical friction in dynamic environments.	Determines human intention through mechanical sliders rather than direct, non-contact optical tracking of human motion and gestures.	Eliminate mechanical contact interfaces by deploying touchless monocular vision tracking combined with planar LiDAR spatial mapping.
Javier Laplaza et al. (2022) [8]	Context and Intention for 3D Human Motion Prediction in Handover Tasks	Attention-based recurrent deep learning model that predicts 3D human reaching motion using motion history, robot spatial pose, and environmental object context.	Experiments restricted to stationary tabletop handover benchmarks and synthetic datasets; does not generalize to mobile navigation spaces.	Confined to localized object handover tasks; lacks integration with mobile platform locomotion or autonomous navigation middleware.	Extend predictive modeling to mobile spatial navigation, coupling short-horizon motion prediction with dynamic vehicle control.
Yongshi Liang and Pai Zheng (2024) [9]	Human Intention Prediction with Visual-Language Model	Utilizes multi-modal Visual-Language Models (VLMs) to infer human intention from combined visual video streams and linguistic prompts in shared spaces.	Extremely high computational footprint requiring multi-GPU server infrastructure; tested in pedestrian traffic without robotic control integration.	Focuses purely on high-level semantic intention prediction; lacks real-time edge execution or physical robotic middleware integration.	Replace computationally heavy VLMs with lightweight, engineered geometric feature extractors deployable on embedded ARM processors.

Authors / Year	Research Topic	Methodology	Limitations	Research Gap	Proposed Improvement
Muhtadin et al. (2025) [7]	Hand Gesture Recognition for Collaborative Robots Using Lightweight Deep Learning in Real-Time Robotic Systems	MediaPipe hand landmark extraction feeding a lightweight artificial neural network, integrated into ROS2 to control a UR5 industrial manipulator arm in real time.	Deployed strictly on a stationary 6-DOF robotic arm; no mobile base, no SLAM mapping, and no spatial obstacle perception.	Demonstrates an edge-feasible gesture pipeline, but does not extend to mobile autonomous robots or multi-modal spatial navigation.	Bridge the lightweight gesture recognition pipeline with mobile chassis kinematics, 2D LiDAR SLAM, and centralized decision state machines.

Chapter 3

System Design and Methodology

3.1 Introduction

This chapter details the methodology, system design, and physical implementation of the real-time, gesture-controlled autonomous mobile robot developed for this study. Moving beyond theoretical simulation, this project is engineered as a fully edge-deployed system. It executes a complex visual perception pipeline natively on onboard embedded hardware, while spatial mapping was validated directly on the physical mobile platform in indoor environments, and the autonomous navigation framework was architecturally integrated for spatial trajectory execution.

The chapter provides a comprehensive breakdown of the system components, progressing from the underlying physical hardware and middleware deployment strategies to the high-level cognitive processes. Specifically, it outlines the architecture of the vision-based intention recognition pipeline—detailing spatial person detection, hand landmark extraction, and geometric feature engineering. Furthermore, it describes the centralised decision-making state machine and the architectural integration of Simultaneous Localization and Mapping (SLAM) for autonomous navigation. By detailing these interconnected subsystems, this chapter documents how human hand gestures were reliably captured, classified, and translated into safe robotic motion within this study.

3.2 System Requirements

Before detailing the system architecture, this section specifies the hardware and software components required to reproduce this system, and the reasoning behind each selection. Component choices were driven by the project’s core requirement of edge-deployed autonomy without reliance on external cloud infrastructure. Both development-time and runtime environments are specified to enable reliable replication.

3.2.1 Hardware Requirements

To achieve fully localized edge cognition, the physical robot was constructed from modular, industrial-grade embedded components. Rather than treating the robot as a monolithic black box, this section details each physical subsystem individually, examining its mechanical role, sensing capabilities, and computational responsibilities within the broader gesture-interaction pipeline. The section concludes with Table 3.1 summarizing the complete hardware specification matrix.

Mobile Robotic Platform and Mechanical Locomotion

The mobile physical foundation of the system is the Yahboom MicroROS-Pi5 differential-drive robotic car, depicted in Figure 3.1. The platform is engineered from an anodized, industrial aluminium-alloy chassis that provides structural rigidity while protecting internal computing modules from mechanical vibrations. Locomotion is driven by four 310 DC geared motors paired with precision metal spur gearboxes. Each motor incorporates a magnetic Hall-effect quadrature encoder that measures wheel rotations with high resolution, providing the foundational tick data necessary for dead-reckoning wheel odometry. The differential-drive kinematic configuration allows the robot to perform smooth forward translations, reverse maneuvers, and zero-radius in-place rotations, ensuring agile maneuverability within tight indoor spaces.



Figure 3.1: Fully Assembled Yahboom MicroROS-Pi5 Autonomous Mobile Robot Platform

Primary Embedded Edge Computer (Raspberry Pi 5)

The robot’s central processing engine is a Raspberry Pi 5 single-board computer equipped with 8GB of high-speed LPDDR4X SDRAM, illustrated in Figure 3.2. Powered by a Broadcom BCM2712 quad-core ARM Cortex-A76 processor operating at 2.4 GHz, this edge computer possesses sufficient floating-point throughput to execute heavy computer vision and machine learning models in real time without offloading computations to external servers. High-bandwidth USB 3.0 interfaces support uncompressed video frame acquisition, while onboard gigabit networking and high-speed Wi-Fi allow live spatial telemetry streaming to remote developer monitoring stations. As established during integration testing (detailed in Section 3.3.3), the 8GB memory configuration is strictly essential to support concurrent execution of laser SLAM mapping and artificial intelligence inference without triggering kernel-level out-of-memory aborts.

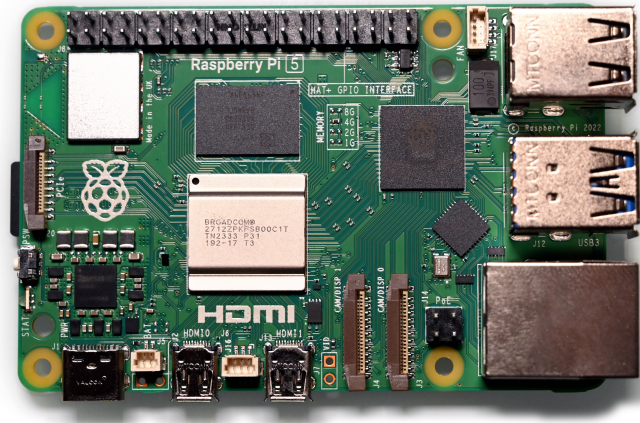


Figure 3.2: Raspberry Pi 5 Single-Board Computer (8GB LPDDR4X Edge Compute Node)

Embedded Microcontroller Co-Processor (MicroROS Expansion Board)

Real-time actuator driving and deterministic sensor sampling are offloaded to a dedicated Yahboom MicroROS robot expansion board, illustrated in Figure 3.3. The board is powered by an Espressif ESP32-S3 dual-core microcontroller running manufacturer FreeRTOS firmware configured as a micro-ROS client. The board integrates an onboard 4-channel H-bridge motor driver capable of delivering continuous high-current Pulse Width Modulation (PWM) signals to the DC drive motors. Additionally, it features a 6-axis MPU6050 Inertial Measurement Unit (IMU) that continuously registers 3-axis angular velocities and 3-axis linear accelerations. By communicating with the Raspberry Pi 5 over a dedicated UART serial connection at 921,600 baud, the expansion board acts as an immutable hardware bridge that isolates real-time motor control loops from high-level vision computing spikes.

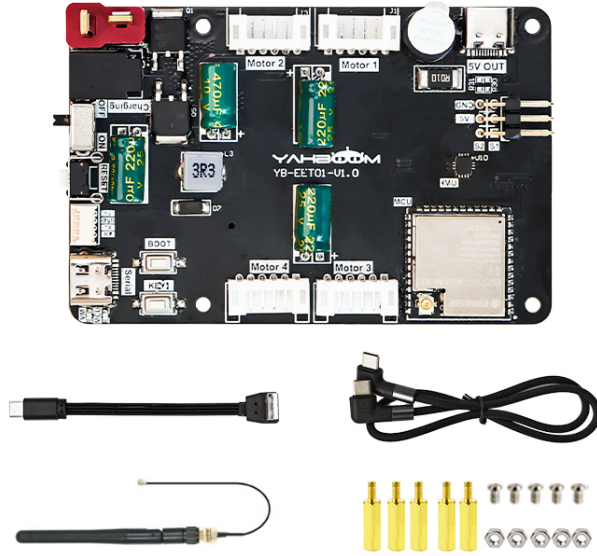


Figure 3.3: Yahboom MicroROS Robot Expansion Board (ESP32-S3 Co-Processor)

Planar Range Sensing (MS200 2D Time-of-Flight LiDAR)

Environmental spatial awareness and collision safety are provided by an MS200 Time-of-Flight (ToF) 2D planar LiDAR sensor, shown in Figure 3.4. Mounted centrally on top of the robot’s upper deck, the LiDAR continuously sweeps a 360-degree horizontal plane at a scan frequency of 12.5 Hz, emitting infrared laser pulses to measure object distances up to a 12-meter radius with millimeter resolution. Unlike traditional triangulation-based optical sensors, the ToF ranging principle provides superior immunity to ambient indoor illumination (up to 30 Klux), ensuring dependable obstacle detection under both bright fluorescent lighting and natural sunlight. In the software architecture, the LiDAR distance array feeds directly into the SLAM Toolbox occupancy mapping pipeline and enforces a hard-coded 0.36 m reactive collision stop.



Figure 3.4: MS200 Time-of-Flight (ToF) 2D Planar LiDAR Sensor

Visual Perception and Active Camera Gimbal

Visual data for human intention recognition is captured using an onboard 2MP USB monocular camera mounted upon a two-degree-of-freedom (2-DOF) pan-tilt servo gimbal mechanism, illustrated in Figure 3.5. The camera features a wide-angle optical lens configured to stream continuous RGB video at 20 frames per second with a resolution of 640×480 pixels. The active gimbal is actuated by two independent digital micro-servos (horizontal pan S1 and vertical tilt S2), enabling the camera to dynamically adjust its viewing orientation. When the robot enters person-tracking or visual-servoing modes, proportional control loops drive the gimbal servos to keep the interacting operator centered within the camera frame, maintaining uninterrupted skeletal hand tracking even as the robot turns or traverses uneven flooring.

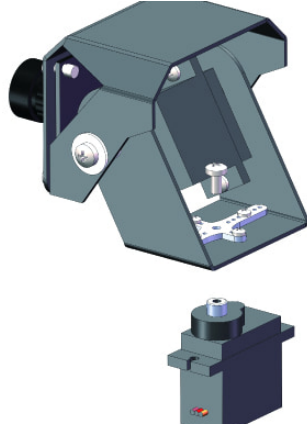


Figure 3.5: 2-DOF Active Pan-Tilt Camera Gimbal Mechanism

Power Distribution, Battery Coupling, and Dual-Bus Regulation

The mobile robot is energized by a 7.4V (nominal) 2000mAh 2-cell (2S) rechargeable lithium-ion battery pack mounted securely within the lower chassis. The battery wiring harness terminates in a keyed male plug that connects directly into the matching female DC power socket soldered onto the Yahboom Micro-ROS expansion board. When fully charged using the dedicated external balance charger, empirical voltage measurements taken across the terminal pins of this board-mounted female connector register above 7.0 V (typically ranging between 7.4 V and 8.2 V, up to an 8.4 V peak open-circuit potential). This direct DC rail supplies the high current demanded by the four DC wheel drive motors during rapid accelerations and continuous locomotion.

Because high-torque DC drive motors draw sharp transient current spikes during rapid acceleration and braking, directly sharing an unregulated power rail with sensitive computing electronics would cause severe voltage brownouts and processor resets. To ensure uninterrupted computation, the power subsystem implements a decoupled dual-bus architecture. While raw battery power feeds directly into the expansion board’s H-bridge motor drivers, an integrated high-efficiency step-down buck converter regulates a clean, isolated 5V/5A power rail dedicated exclusively to powering the Raspberry Pi 5 single-board computer and its digital sensors. In this mechatronic topology, the Raspberry Pi 5 functions as the central Edge Computing Hub, orchestrating high-level visual cognition, multi-node ROS2 communication, and spatial mapping while remaining electrically insulated from motor-induced inductive noise.

Figure 3.6 depicts the physical mechatronic layout and wiring harness of the robot

undergoing validation on the laboratory test bench prior to top casing installation. The stacked embedded computing architecture is clearly demonstrated, featuring the Raspberry Pi 5 mounted directly above the Yahboom MicroROS expansion board to minimize interconnect cable lengths. The dedicated wiring harness separates high-current motor power cables from low-voltage logic signal lines, while multi-conductor ribbon cables route quadrature encoder feedback and servo PWM lines directly into the ESP32-S3 microcontroller co-processor.

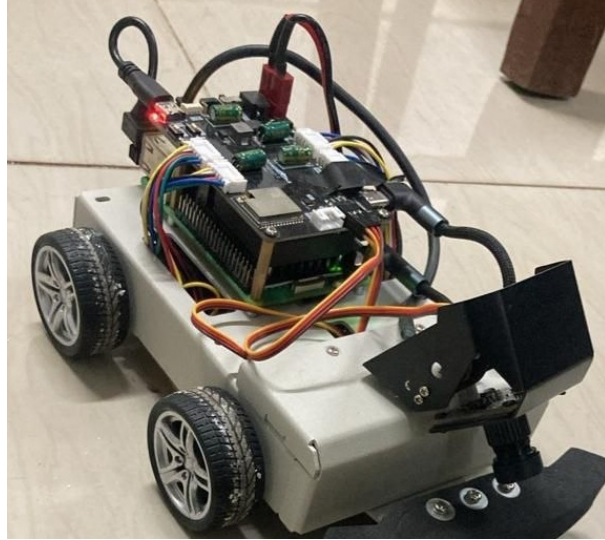


Figure 3.6: Internal Chassis Hardware Integration and Mechatronic Wiring Harness (Uncased View)

Consolidated Hardware Requirements Summary

Table 3.1 consolidates the technical specifications, physical parameters, and operational roles of all hardware components detailed in the preceding subsections. By establishing explicit performance thresholds for compute power, sensor range, and communication interfaces, this hardware baseline guarantees deterministic execution of both edge neural networks and real-time motor control loops. The complete Bill of Materials (BOM) integrates standard industrial-grade robotics modules, balancing high operational reliability against edge power consumption constraints.

Table 3.1: Hardware Components, Technical Specifications, and Functional Roles

Component	Technical Specification	Project Functionality & Operational Role
Primary Computing Unit	Raspberry Pi 5, 8GB LPDDR4X SDRAM, quad-core ARM Cortex-A76 @ 2.4 GHz	Executes high-level perception pipeline (YOLOv8n spatial filter, MediaPipe hand tracker, MLP classifier, LSTM path predictor) and hosts Docker containers.
Robot Expansion Board	Custom ESP32-S3 co-processor; 4-channel motor driver; 2-channel servo PWM; 6-axis MPU6050 IMU	Operates micro-ROS client firmware; converts ROS2 velocity commands into motor PWM signals; streams encoder ticks and IMU angular velocity at 921,600 baud.
Mobile Chassis & Locomotion	Yahboom industrial aluminium-alloy differential-drive platform; 4× 310 DC geared motors	Provides physical mobile locomotion; executes linear and angular motion trajectories; transmits quadrature wheel encoder feedback for dead-reckoning odometry.
Planar LiDAR Sensor	MS200 Time-of-Flight (ToF) 2D LiDAR; 12.5 Hz scan rate; 12 m range; 360° field of view	Delivers 2D laser range scans to <code>/scan</code> topic for SLAM Toolbox occupancy grid mapping; provides distance triggers for the 0.36 m reactive collision halt.
Monocular Vision Sensor	2MP USB monocular CMOS camera (1080p @ 30 FPS, operated at 640×480 @ 20 FPS)	Captures continuous frontal RGB video stream for person detection, spatial acceptance zone gating, hand cropping, and visual servoing tracking.
Active Camera Gimbal	2-DOF pan-tilt servo mechanism with anti-windup angle clamping	Continuously tracks the operator’s centroid; centers the human subject in the camera frame to maintain reliable visual gesture tracking during robot turns.
Power Supply & Regulation	7.4V 2000mAh rechargeable Li-ion battery pack with 5V/5A step-down buck converter	Supplies dual decoupled power rails: a regulated 5V rail for Raspberry Pi 5 logic and a direct rail for motor driver surges, preventing voltage drops and brownouts.

3.2.2 Software Requirements

Table 3.2 itemizes the software stack, operating systems, and robotics frameworks utilized across the development and physical deployment phases. In robotics engineering, software is typically divided between the “developer workstation” where intensive graphics, simulations, and code development take place, and the “mobile robot” where lightweight, battery-efficient programs execute in real time. This separation ensures that heavy visualization tasks—such as viewing live 3D laser maps—do not drain the robot’s onboard processor or interfere with its physical movement decisions.

Table 3.2: Software Components, Deployment Targets, and System Roles

Component	Deployment Target	System Role
Ubuntu 24.04 LTS	Development Workstation	Host operating system for code development, neural network training, and simulation.
ROS2 Jazzy Jalisco	Development Workstation	Development middleware used for 3D spatial visualization (RViz2) and simulated testing.
Ubuntu 20.04 LTS	Robot (Docker Container)	Containerized Linux base environment hosting ROS2 Humble and hardware driver stacks.
ROS2 Humble Hawksbill	Robot (Docker Container)	Deployed robotics middleware providing reliable inter-process messaging via DDS transport.
Docker Engine 26.1	Robot (Raspberry Pi 5)	Virtualization engine providing isolated, reproducible software containers on the mobile host.
MediaPipe Hand-Landmarker	Robot (yahboom_gesture)	Vision model that tracks 21 anatomical landmarks on the human hand in real time.
Ultralytics YOLOv8n	Robot (yahboom_gesture)	Lightweight neural network for detecting human bodies and filtering out background bystanders.
ONNX Runtime 1.18	Robot (yahboom_gesture)	High-performance cross-platform engine executing gesture classifier and trajectory models.
scikit-learn (joblib)	Robot (yahboom_gesture)	Machine learning utility library for feature transformation and decoding model prediction labels.
SLAM Toolbox	Robot (yahboom_base)	Laser-based mapping software that generates 2D indoor floor plans as the robot drives.
Navigation2 (Nav2)	Robot (yahboom_gesture)	Autonomous navigation suite for waypoint path planning and obstacle-avoidance costmaps.
micro-ROS Client	ESP32-S3 Microcontroller	Real-time embedded firmware running on FreeRTOS; directly regulates motor PWM and sensors.
Python 3.10 / 3.12	Both Targets	Primary programming language used

This architectural split reflects the resource-planning strategy detailed in Section 3.3.3. Development convenience on the workstation is cleanly decoupled from the strict execution requirements of the physically deployed robot, with containerization providing an immutable software boundary. The remote workstation allows human supervisors to observe the robot’s camera feed and spatial maps comfortably over a wireless network without stealing memory or computational power from the robot itself.

3.3 System Architecture Overview

The system architecture of the autonomous mobile robot is structured around a multi-tiered hierarchy that clearly separates high-level cognitive perception from low-level physical actuation. In intuitive terms, this mirrors biological motor control: a central brain processes visual sights and formulates intentions, while a dedicated spinal reflex system translates those intentions into smooth physical muscle movements. By separating high-level neural inference from real-time motor pulse generation, each processor operates within its optimal performance limits without compromising mechanical responsiveness or physical safety.

Figure 3.7 illustrates the overarching end-to-end system architecture and cognitive signal flow across the four primary operational tiers of the robot. Moving beyond isolated software components, this unified pipeline establishes deterministic intention-to-action coupling on edge hardware. Progressing sequentially from environmental sensing to mechanical wheel actuation, the pipeline coordinates sensory acquisition, spatial filtering, feature extraction, and motion arbitration across the embedded compute modules:

1. **Tier 1: Multi-Modal Environmental Sensing:** The sensory layer captures simultaneous visual and range data from the surroundings. The monocular USB camera captures wide-angle frontal imagery at a resolution of 640×480 pixels operating at 20 frames per second (FPS), while the MS200 Time-of-Flight (ToF) LiDAR continuously sweeps a 360-degree horizontal plane at 12.5 Hz measuring distances from 0.12 m up to 12.0 m. Simultaneously, the onboard 6-axis IMU continuously samples chassis angular rates to support dead-reckoning odometry.
2. **Tier 2: Spatial Perception and Operator Gating:** The perception layer filters the incoming video stream using a YOLOv8n spatial bound-

ing box detector to isolate valid human bodies. Rather than processing the entire visual scene indiscriminately, a central spatial acceptance zone (spanning 45% of the frame width and 65% of the frame height) isolates the intended operator, rejecting peripheral bystanders. This stage computes the horizontal tracking error ($e_x = C_x - 320$) relative to the camera frame center to guide continuous visual servoing.

3. **Tier 3: Feature Extraction and Gesture Classification:** For the isolated operator, MediaPipe HandLandmarker extracts 21 skeletal landmarks in real time. These coordinates are mathematically converted into a compact 19-dimensional scale-invariant geometric feature vector that is evaluated by a trained Multilayer Perceptron (MLP) running on the ONNX Runtime engine, completing inference in under 2 milliseconds.
4. **Tier 4: Supervisory Control and Actuation:** The cognitive decision-making node (`brain_node`) maintains a rolling consensus buffer and supervisory state machine to arbitrate driving commands. Validated velocity vectors (`/cmd_vel`) are transmitted across the micro-ROS serial bridge at 921,600 baud to the ESP32-S3 co-processor, which directly generates motor PWM signals.

In addition to this four-tier sequential cognition pipeline, Figure 3.7 highlights an essential hard-wired safety mechanism: the planar LiDAR reactive safety bypass. If the MS200 LiDAR detects any physical obstacle within a critical 0.36 m protective safety bubble directly in front of the vehicle, the safety controller immediately overrides all active gesture commands and commands a zero-velocity halt. This reactive interlock line operates independently of visual perception and machine learning inference delays, ensuring guaranteed collision prevention in compliance with collaborative robot safety principles.

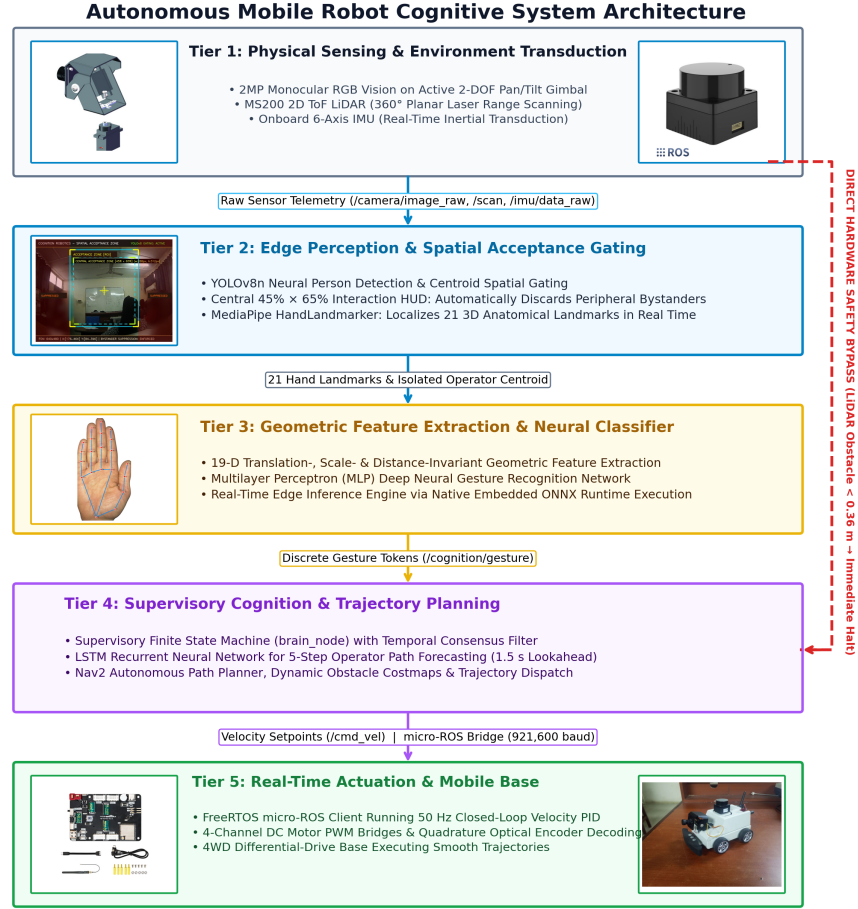


Figure 3.7: End-to-End System Architecture and Cognitive Signal Flow

3.3.1 Hardware Platform Specification

The physical foundation of the system is the Yahboom MicroROS-Pi5 differential-drive mobile platform, an industrial aluminium-alloy robotic base designed for edge intelligence research. To achieve complete autonomy without relying on remote cloud servers or high-latency internet connections, the primary computing engine is a Raspberry Pi 5 single-board computer. This onboard computer functions as the primary edge computing node, executing vision models, neural network classifiers, and spatial navigation software entirely on edge hardware.

Low-level hardware management, power distribution, and motor driving are handled by a custom robotics expansion board powered by an ESP32-S3 micro-controller. This co-processor executes deterministic real-time motor commutation and low-level sensor sampling, interfacing directly with the four DC wheel motors, reading high-speed wheel rotation sensors (quadrature encoders), and polling an onboard 6-axis Inertial Measurement Unit (IMU). By offloading these

time-sensitive hardware tasks to the ESP32-S3, the robot maintains steady wheel balance and responsive steering even when the main processor is busy running heavy vision algorithms.

For environmental awareness, the system utilizes an MS200 2D Time-of-Flight (ToF) LiDAR sensor mounted centrally on top of the chassis, emitting laser pulses to measure distances in a 360-degree flat circle 12.5 times per second. Visual perception for human interaction is provided by a 2MP monocular USB camera mounted on an active two-axis (pan-tilt) servo gimbal. This motorized gimbal provides two degrees of rotational freedom to maintain optical alignment with an interacting human operator, panning horizontally and tilting vertically to keep the human subject centered within the focal frame. Figure 3.8 illustrates the physical hardware interconnection and power distribution topology.

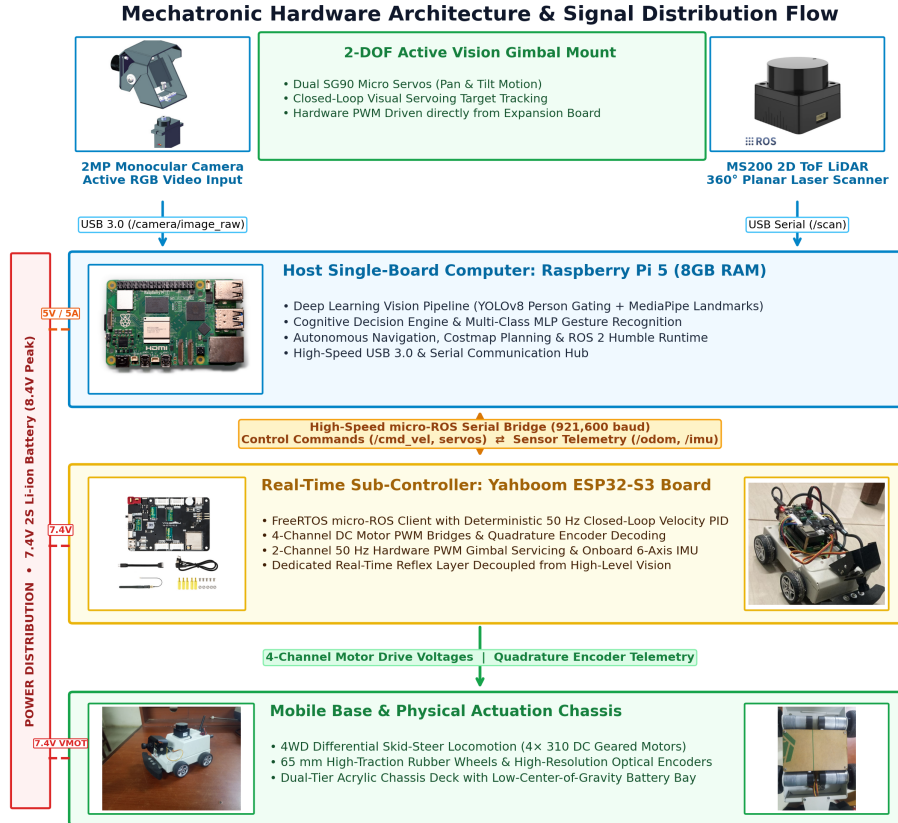


Figure 3.8: Hardware System Interconnection and Power Topology

3.3.2 Software Framework and Deployment Strategy

The software architecture is built natively on the Robot Operating System 2 (ROS2 Humble Hawksbill distribution) [10]. ROS2 provides an asynchronous,

distributed middleware framework enabling modular software nodes to exchange sensor telemetry and motion commands across standardized topic interfaces. To achieve optimal performance, high-level perception nodes are written in Python to leverage powerful machine learning libraries, whereas low-level motor drivers rely on the ESP32-S3’s lightweight FreeRTOS real-time firmware.

Because small microcontrollers like the ESP32-S3 do not have enough memory to run a full Linux operating system, the board runs a specialized protocol called micro-ROS. This micro-ROS client communicates over a high-speed serial cable (operating at 921,600 baud) with a micro-ROS agent running on the Raspberry Pi, seamlessly integrating the microcontroller into the broader ROS2 network. This bridge allows the microcontroller to publish sensor data and receive driving instructions using standard ROS2 messages. By isolating time-critical motor control on the microcontroller, the robot prevents physical wheel stuttering if the main vision processor experiences a momentary computational spike.

To ensure software components run reliably without conflicting dependencies, all onboard software is organized into isolated virtual environments called Docker containers, enforcing strict process isolation and dependency encapsulation. This containerized strategy creates clean software boundaries, ensuring that third-party hardware drivers remain isolated from custom machine learning packages. The software architecture is partitioned across three distinct containers on the Raspberry Pi 5:

1. **yahboom_base**: Houses low-level hardware drivers, managing serial port communication, LiDAR scanning, and base chassis telemetry.
2. **micro_ros_agent**: Serves as the real-time translation gateway between the ESP32-S3 microcontroller and the ROS2 messaging bus.
3. **yahboom_gesture**: Contains all custom cognition, computer vision, human detection, gesture classification, and decision-making nodes engineered in this study.

All three containers are built upon Yahboom’s official base image (`yahboomtechnology/ros-hum`) with the custom perception and control nodes layered cleanly on top within the **yahboom_gesture** container. This layering strategy preserves tested factory hardware drivers while isolating newly developed artificial intelligence code in an easily modifiable user layer. As a result, experimental perception models can be revised and re-deployed in seconds without risking the stability of core chassis safety drivers.

All three containers are launched in Docker’s **host** network mode, allowing them to share the host network stack directly rather than communicating through virtual network bridges. This direct communication allows ROS2’s built-in discovery service to link nodes across container boundaries with sub-millisecond latency. Crucially, all containers and host drivers must share an identical communication domain setting known as the **ROS_DOMAIN_ID**. During early system integration, an elusive communication failure occurred when the micro-ROS hardware daemon and base driver container operated on domain 20, whereas custom perception nodes initialized on the default domain 0. This mismatch caused the robot to silently drop motor commands across the container boundary until all environments were standardized to **ROS_DOMAIN_ID=20**.

Finally, to make the robot fully self-sufficient upon battery power-up, all software containers are automatically initialized by an operating system background service (**cognition.service**). This ensures that the robot immediately begins listening for human gestures the moment the physical power switch is flipped, without requiring manual keyboard input. During testing, the robot operates headlessly without a monitor to conserve battery and memory, while developers monitor spatial maps and diagnostic cameras remotely over Wi-Fi using RViz2 on an **x86_64** engineering host workstation (AMD Ryzen 5, 8GB DDR4 RAM running Ubuntu 24.04 LTS and ROS2 Jazzy Jalisco). Figure 3.9 illustrates this containerized multi-node deployment architecture.

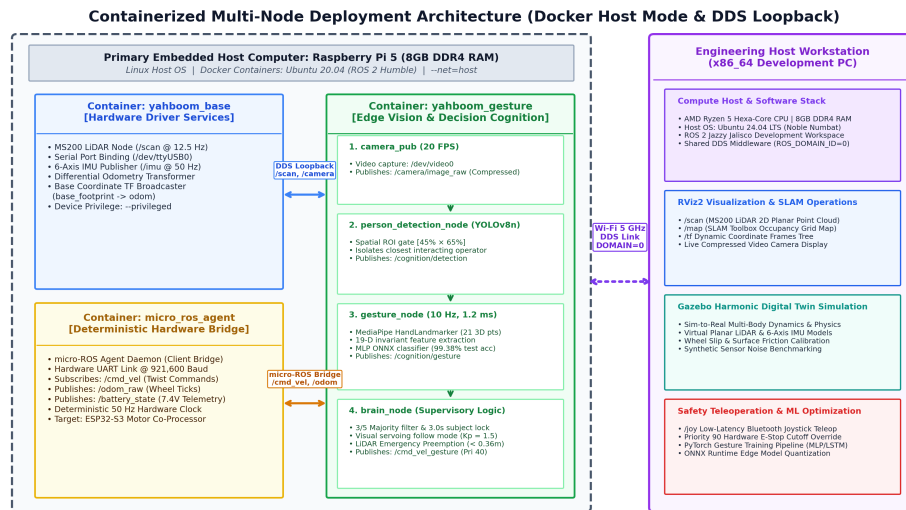


Figure 3.9: Containerized Multi-Node Deployment Architecture (Docker Host Mode)

3.3.3 Hardware Resource Planning and Memory Constraints

During initial system prototyping, the platform was evaluated using a Raspberry Pi 5 equipped with 2GB of RAM, based on the preliminary hypothesis that individual perception and motion nodes would operate within a modest memory footprint. However, concurrent execution of the SLAM Toolbox laser mapping node alongside the full perception pipeline—comprising YOLOv8n spatial filtering, MediaPipe HandLandmarker keypoint extraction, and MLP neural inference—frequently exhausted physical memory, resulting in `std::bad_alloc` exceptions. Under this constrained 2GB ceiling, critical ROS2 communication queues and costmap buffers saturated available physical memory, causing abrupt cessations of the `/scan` and camera video topics midway through live trials.

Diagnostic investigation confirmed that these process failures were triggered by the Linux Out-Of-Memory (OOM) killer terminating memory-intensive nodes to protect kernel execution. When the occupancy grid mapping array and neural network runtimes executed concurrently, aggregate memory consumption exceeded the physical capacity of the 2GB board. The resulting kernel terminations caused sudden communication drops across the inter-process message bus.

To resolve this bottleneck permanently, the physical hardware was upgraded to an 8GB LPDDR4X Raspberry Pi 5 single-board computer. This hardware upgrade eliminated memory allocation failures entirely, providing sufficient capacity to execute 2D laser SLAM, deep neural network inference, supervisory state logic, and telemetry logging simultaneously with over 3.8 GB of unallocated physical memory headroom remaining. Documenting this memory constraint is vital for engineering reproducibility, as it established the operational resource baseline and justified the Quality-of-Service (QoS) message queue depths implemented across the communication graph.

3.4 Data Acquisition and Dataset Preparation

To ensure the Multilayer Perceptron (MLP) gesture classifier performed reliably during physical deployment, a custom dataset was engineered specifically for this study. Utilizing pre-existing public gesture datasets often introduces a severe domain shift error, where external image corpora captured via desktop webcams fail to represent the low-angle perspective, wide-angle lens distortion, and optical characteristics of the robot’s physical mounting. In contrast, an autonomous

mobile robot interacts from a low-elevation optical vantage point subject to varying indoor lighting and background activity. By acquiring all training samples directly through the robot’s onboard optical sensor, the neural network learned representations that accurately reflect the focal geometry, mounting height, and lens distortion encountered during live physical trials.

3.4.1 Recording Conditions and Class Balancing

The dataset was constructed to encompass the six explicit operational commands: STOP, GO, LEFT, RIGHT, BACK, and FOLLOW. To ensure the neural network learned the geometric relationships of the hand rather than memorizing the background, data collection was conducted across three distinct indoor environments with varying conditions of natural daylight and artificial fluorescent lighting. Operators were positioned at realistic operational distances ranging between 1.0 m and 2.5 m from the robot.

To prevent model bias toward any specific gesture, the dataset was strictly balanced. Exactly 1,000 distinct frames were recorded for each of the six gesture classes, resulting in a total dataset of 6,000 labeled instances. During recording, the operator performed micro-variations of each gesture (altering wrist roll, pitch, and yaw within a 15-degree tolerance) to ensure the model generalized effectively to natural, imperfect human inputs.

3.4.2 Preprocessing and Train-Test Splitting

The raw captured frames were not fed directly into the model. To mirror the live inference pipeline exactly, the raw dataset was processed through the YOLOv8n spatial filter to generate bounding boxes, followed by the MediaPipe HandLandmarker to extract the 21 anatomical coordinates. Quality assurance filtering automatically rejected frames where extreme motion blur or severe physical occlusion prevented MediaPipe from achieving minimum landmark tracking confidence (≥ 0.50), ensuring high signal integrity across the training set. The extracted coordinates were then mathematically converted into the 19 normalized geometric features.

Prior to training, the finalized 6,000-sample feature dataset was randomly shuffled and partitioned using a standard machine learning distribution ratio: 70% (4,200 samples) allocated for training the model, 15% (900 samples) reserved for contin-

uous validation during training, and 15% (900 samples) isolated as a blind test set to evaluate final classification accuracy. This strict partitioning ensured that the model’s reported test performance was evaluated entirely on unseen samples. Isolating a distinct 15% holdout partition ensured that model generalization was rigorously evaluated on unseen geometric feature distributions, guarding against empirical overfitting and confirming that the classifier learned robust decision boundaries across all operational gestures.

3.4.3 Dataset Statistics

This section provides a detailed tabular breakdown of the gesture dataset described in Section 3.4.1 and Section 3.4.2. Table 3.3 presents the per-class distribution across the 6,000 recorded frames, while Table 3.4 details the train, validation, and test partitions. Maintaining an identical quota of 1,000 samples for each command prevented artificial class bias, guaranteeing that the robot would not favor common gestures (such as STOP) over rotational steering commands (such as LEFT or RIGHT).

Table 3.3: Per-Class Sample Distribution

Gesture Class	Samples	Share of Dataset
STOP	1,000	16.7%
FOLLOW	1,000	16.7%
GO	1,000	16.7%
LEFT	1,000	16.7%
RIGHT	1,000	16.7%
BACK	1,000	16.7%
Total	6,000	100.0%

Table 3.4: Train / Validation / Test Split

Split	Samples	Proportion
Training	4,200	70.0%
Validation	900	15.0%
Test	900	15.0%
Total	6,000	100.0%

Data collection spanned three distinct indoor environments with varying natural and artificial fluorescent lighting conditions. Micro-variations in wrist roll, pitch,

and yaw (within a 15-degree tolerance) were systematically introduced during recording to improve model robustness. This balanced structure provided a stable foundation for training the gesture classifier.

3.5 Perception and Intention Recognition Pipeline

The perception pipeline was designed as a sequential filter to isolate and classify human hand gestures from raw visual data. This architecture was specifically optimized for localized inference, allowing the system to extract precise features and execute classification models natively on the edge processor. The pipeline isolated a valid human operator, extracted anatomical hand landmarks, engineered scale-invariant geometric features, and classified the intended gesture.

3.5.1 Spatial Zone Person Detection

The first stage of the perception pipeline utilized the YOLOv8n (nano variant) object detection model [11] to detect human presence in the camera field of view. During initial development and empirical testing, a critical vulnerability was observed: the downstream skeletal tracking framework would unpredictably extract landmarks from any visible hand in the camera frame. In environments with multiple individuals, the robot would erroneously lock onto background movements or gestures performed by non-operators, leading to erratic system behavior.

To resolve this interference and enforce operational safety, a spatial zone filter was engineered over the raw YOLOv8n bounding box outputs. On the standard 640×480 pixel video frame, the camera’s field of view was programmatically partitioned into a central active acceptance zone spanning 45% of the frame width (288 pixels) and 65% of the frame height (312 pixels). This defines an explicit horizontal acceptance window of $X \in [176, 464]$ and a vertical window of $Y \in [84, 396]$ centered at $(320, 240)$. For every detected human candidate, the system evaluates the 2D bounding box coordinates $[x_{\min}, y_{\min}, w, h]$ and calculates the horizontal and vertical centroid (B_x, B_y) as defined in Equation (3.1):

$$B_x = x_{\min} + \frac{w}{2}, \quad B_y = y_{\min} + \frac{h}{2} \quad (3.1)$$

Evaluating this spatial centroid ensures that the operator is actively facing the robot and centered within its immediate operational corridor before any down-

stream skeletal landmark extraction or gesture interpretation takes place. By referencing the candidate’s center of mass against this predefined spatial boundary, spurious peripheral movements are ignored before consuming CPU cycles. This geometric constraint provides a robust first line of defense against false activations in busy collaborative workspaces.

Figure 3.10 illustrates this empirical spatial receptive zone perception and the accompanying ISO 5807 decision flowchart, contrasting the active operator targeting box against peripheral bystanders who are filtered out and protected by Gaussian privacy blurring. In accordance with the decision logic, a candidate detection is accepted as an operator only if its centroid satisfies $176 \leq B_x \leq 464$ and $84 \leq B_y \leq 396$. If multiple persons occupy the camera frame, the system evaluates up to three candidates, selecting the largest (closest) zone-eligible bounding box as the active operator. When an operator issues a gesture that passes the 5-frame consensus threshold described in Section 3.6.2, the **brain_node** establishes a 3-second operational lock, during which background bystanders outside the central zone are rejected from interfering with the robot’s navigation path.

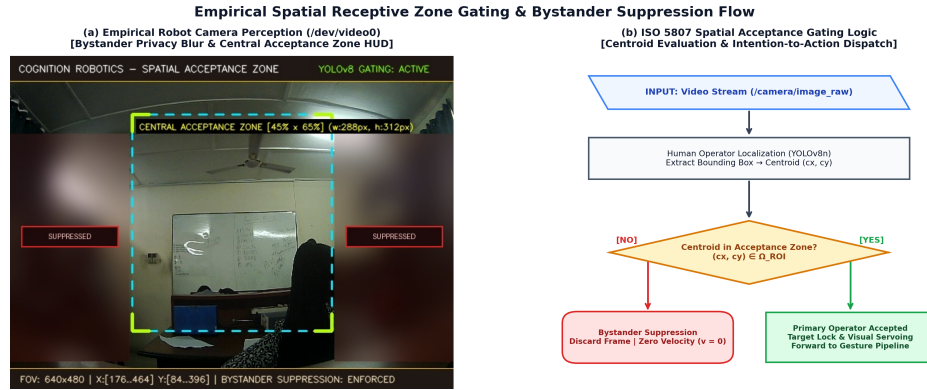


Figure 3.10: Empirical Spatial Receptive Zone Gating and Bystander Suppression Flow

This geometric spatial zone filter relies on horizontal bounding-box coordinates to isolate the primary operator, prioritizing low computational overhead during real-time interaction. In crowded environments where multiple individuals enter the central zone concurrently, spatial heuristic disambiguation defaults to selecting the candidate with the largest projected bounding box area. To evaluate unique individual identification, a biometric facial recognition pipeline was developed using InsightFace (buffalo_sc MobileFaceNet generating 512-dimensional ArcFace embeddings). Offline verification confirmed clear separation between enrolled operators (cosine similarity 0.54–0.57) and unauthorized individuals (0.09). Because running concurrent deep face detection on the edge CPU would constrain

gesture inference latency, continuous biometric authentication was preserved as a modular architecture targeted for dedicated onboard NPU acceleration in future iterations.

3.5.2 Vision-Based Hand Tracking

Once an operator was successfully isolated within the spatial zone, the pipeline engaged the MediaPipe HandLandmarker framework [12] for skeletal tracking. This framework extracted high-fidelity anatomical data from the 2D monocular camera feed. MediaPipe fits a 2.5D topological skeletal model to the localized hand region, outputting 21 distinct anatomical landmarks corresponding to the wrist, metacarpophalangeal, interphalangeal, and fingertip joints in real time.

For each valid frame containing the primary operator, the node identified the spatial coordinates of 21 specific hand landmarks, including the wrist, knuckles, and fingertips. Each landmark was output as a normalized three-dimensional coordinate (x, y, z) . While this provided a detailed topological representation of the hand, feeding these raw, unscaled coordinates directly into a lightweight classifier introduced significant statistical variance depending on the operator’s physical distance from the camera. Consequently, these raw landmarks served strictly as the foundational input for the subsequent geometric feature engineering stage.

3.5.3 Geometric Feature Engineering

To ensure the classification model remained robust regardless of the operator’s distance from the camera, the raw spatial landmarks were transformed into a set of scale-invariant geometric features. In computer vision, an object close to the lens appears large on screen, while the same object positioned several meters away appears tiny, even though its physical geometry has not changed. If a neural network attempts to classify gestures using raw pixel coordinates, it easily confuses changes in distance with changes in hand shape. To overcome this limitation, the raw anatomical coordinates were transformed into scale-invariant geometric features—calculating relative angles and normalized joint ratios that remain identical whether the operator stands one meter or three meters away from the robot.

The feature extraction pipeline executes three sequential mathematical opera-

tions: origin translation, scale normalization, and joint angle derivation. First, to ensure invariance against hand translation within the camera frame, the wrist landmark ($\mathbf{p}_0$) is designated as the local coordinate origin $(0, 0, 0)$. All twenty remaining hand landmark coordinates ($\mathbf{p}_i = [x_i, y_i, z_i]^T$) are translated relative to this origin as defined in Equation (3.2):

$$\mathbf{p}'_i = \mathbf{p}_i - \mathbf{p}_0, \quad \text{for } i \in \{0, 1, \dots, 20\} \quad (3.2)$$

By referencing all twenty remaining anatomical keypoints directly to the wrist origin, the resulting coordinate structure becomes completely immune to the hand's absolute Cartesian position in the camera frame. In mathematical terms, this shifts the coordinate system from camera space to hand-centric space. Consequently, an operator holding their hand high in the frame produces an identical coordinate representation to one holding their hand low.

Second, to achieve scale invariance across varying operating distances, the translated coordinates are normalized by the operator's anatomical palm width (W_{palm}). In physical human anatomy, the distance across the knuckles remains constant regardless of the viewer's vantage point. The palm width is computed as the 3D Euclidean distance between the index metacarpophalangeal joint (Landmark 5) and the pinky metacarpophalangeal joint (Landmark 17) using Equation (3.3):

$$W_{\text{palm}} = \|\mathbf{p}'_5 - \mathbf{p}'_{17}\| = \sqrt{(x'_5 - x'_{17})^2 + (y'_5 - y'_{17})^2 + (z'_5 - z'_{17})^2} \quad (3.3)$$

To scale each coordinate proportionally, each translated landmark and fingertip distance is subsequently divided by this reference palm width. This normalization ensures that visual features reflect true anatomical proportions rather than distance-dependent pixel sizes. The transformation is computed per Equation (3.4):

$$\hat{\mathbf{p}}_i = \frac{\mathbf{p}'_i}{W_{\text{palm}}} \quad (3.4)$$

This proportional division guarantees that the resulting numerical values remain constant whether the user interacts near the camera lens or several meters away in the room. Even under significant forward-backward motion, the normalized coordinates maintain consistent numerical bounds. This eliminates the need to train separate distance-dependent gesture models.

Third, to determine whether individual fingers are curled or extended independently of hand tilt in space, the system derives 3D cosine joint curl angles. For each of the five digits, vector $\vec{u}$ represents the bone segment between the metacar-

popphalangeal (MCP) and proximal interphalangeal (PIP) joints, while vector $\vec{v}$ represents the segment between the PIP and fingertip (TIP) joints. The curl angle (θ_j) is calculated using the vector dot product formulation in Equation (3.5):

$$\theta_j = \arccos \left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|} \right) \quad (3.5)$$

When a finger is extended straight outward, the angle θ_j approaches zero radians (0°). As the finger curls tightly into the palm, the angle smoothly increases toward π radians (180°). As summarized in Table 3.5, combining these normalized fingertip distances, finger curl angles, inter-finger spread angles, and thumb spatial depth yields a compact, 19-dimensional feature vector.

Table 3.5: Engineered Geometric Features (19 total)

#	Feature	Description
1	Thumb curl angle	Angle between MCP–PIP and PIP–TIP segments
2	Index curl angle	Angle between MCP–PIP and PIP–TIP segments
3	Middle curl angle	Angle between MCP–PIP and PIP–TIP segments
4	Ring curl angle	Angle between MCP–PIP and PIP–TIP segments
5	Pinky curl angle	Angle between MCP–PIP and PIP–TIP segments
6	Thumb–Index spread	Angle between thumb and index tip vectors (from wrist)
7	Index–Middle spread	Angle between index and middle tip vectors (from wrist)
8	Middle–Ring spread	Angle between middle and ring tip vectors (from wrist)
9	Thumb tip distance	Thumb tip–to–wrist distance, normalized by palm width
10	Index tip distance	Index tip–to–wrist distance, normalized by palm width
11	Middle tip distance	Middle tip–to–wrist distance, normalized by palm width
12	Ring tip distance	Ring tip–to–wrist distance, normalized by palm width
13	Pinky tip distance	Pinky tip–to–wrist distance, normalized by palm width
14	Palm angle	Angle of middle-MCP–to–wrist vector vs. vertical reference
15	Index pointing angle	2D angle of the index finger’s direction vector
16	Thumb pointing angle	2D angle of the thumb’s direction vector
17	Index–Thumb angle	Angle between index and thumb vectors (from wrist)
18	Thumb–Index tip distance	Distance between thumb tip and index tip, normalized
19	Thumb height	Thumb tip depth (z-axis) relative to wrist

3.5.4 Gesture Classification Model

An initial implementation of the gesture classification subsystem utilized a rule-based heuristic classifier, applying static geometric thresholds directly to the 19 engineered features. While functional for a small set of orthogonal hand configurations, this deterministic heuristic approach proved brittle during dynamic interaction. Small variations in hand orientation, distance from the camera, and natural inter-operator biomechanical differences frequently caused feature vectors to oscillate across hand-tuned boundaries, requiring continuous manual recalibration. This operational brittleness motivated the transition to a supervised Multilayer Perceptron (MLP) classifier, which learns continuous, non-linear decision boundaries across empirical data distributions, generalizing effectively across natural morphological variations in human gesture execution.

With the spatial data reduced to 19 scale-invariant features, the final stage of the perception pipeline was the classification of the intentional command using this MLP. The neural network was architected as a compact feedforward model consisting of four fully-connected layers: an input layer of 19 feature nodes, a primary hidden layer of 128 neurons, a secondary hidden layer of 64 neurons, and an output layer of 6 classification neurons ($19 \rightarrow 128 \rightarrow 64 \rightarrow 6$). The hidden layers incorporate Rectified Linear Unit (ReLU) activation functions ($\text{ReLU}(z) = \max(0, z)$) to capture complex non-linear combinations of finger flexions, while the final layer employs a Softmax activation function to generate normalized posterior probability distributions across the six discrete commands: STOP, GO, LEFT, RIGHT, BACK, and FOLLOW. By minimizing structural parameter depth, the forward inference pass executes in under 2 milliseconds on the Raspberry Pi 5 CPU via the ONNX Runtime engine.

The model development protocol comprised two iterative training cycles. An initial preliminary dataset of 3,000 samples yielded an initial test accuracy of 99.67%; however, class representation was skewed toward static poses. To establish rigorous cross-class generalization and eliminate classification bias, the dataset was expanded to a fully balanced distribution of 6,000 samples (1,000 samples per class across six distinct commands). On this standardized holdout benchmark, the MLP classifier achieved a 99.38% test accuracy, demonstrating uniform class sensitivity and robust geometric discrimination across all six operational commands.

The final trained classifier was exported to Open Neural Network Exchange (ONNX) format (`gesture_model.onnx`) and loaded via ONNX Runtime (`onnxruntime.InferenceSession`).

inside `gesture_node` at runtime, with a separate `label_encoder_pi.pkl` file—loaded via scikit-learn’s `joblib`—handling decoding of the model’s integer output back to a gesture label string. ONNX Runtime was used consistently across both learned models in the perception pipeline, providing a uniform inference approach for the gesture classifier and the LSTM path predictor alike. The raw classification output was then published to the ROS2 middleware, where it was intercepted by the centralized decision-making node. Figure 3.11 illustrates this complete four-stage pipeline: starting from the 21 MediaPipe anatomical hand landmarks extracted from empirical camera data (P_0 through P_{20}), progressing through scale and translation invariance transforms, constructing the 19-dimensional invariant geometric feature vector, and terminating in the low-latency Multilayer Perceptron classifier generating discrete action tokens.

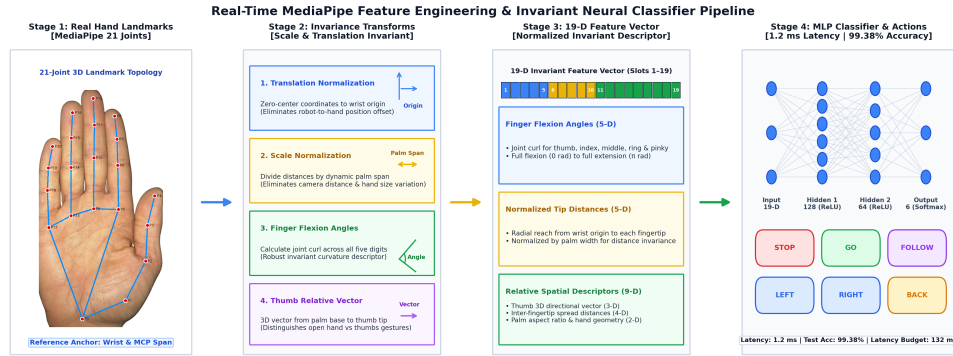


Figure 3.11: Real-Time MediaPipe Feature Engineering and Invariant Neural Classifier Pipeline

3.5.5 Human Movement Prediction (LSTM Path Predictor)

In addition to static hand-gesture classification, a secondary recognition pathway was implemented to capture human movement intent over time, complementing the frame-by-frame gesture pipeline with a temporal model of operator motion. Sequential pose landmarks extracted via MediaPipe Pose were fed into a Long Short-Term Memory (LSTM) network trained to predict an operator’s short-horizon future position, allowing the system to anticipate movement direction rather than react only after a position change had already occurred. Natural human locomotion exhibits continuous kinematic dependencies where intentional heading transitions and velocity shifts unfold progressively over time. By modeling sequential keypoint trajectories over a sliding temporal window of the preceding 30 frames (1.0 second), the LSTM network forecasts the operator’s spatial

trajectory 0.5s into the future, enabling the mobile robot to initiate proactive steering adjustments and smooth acceleration profiles that mitigate phase lag inherent in purely reactive tracking controllers.

The model was trained and evaluated on 152 motion sequences, achieving an Average Displacement Error (ADE) of 25.8 pixels and a Final Displacement Error (FDE) of 32.4 pixels on held-out test sequences. The trained model was exported to ONNX format, consistent with the deployment strategy used for the gesture classifier, to ensure uniform inference performance on the Raspberry Pi 5's edge processor. Figure 3.12 illustrates the training loss convergence, and Figure 3.13 depicts representative predicted trajectories against ground truth.

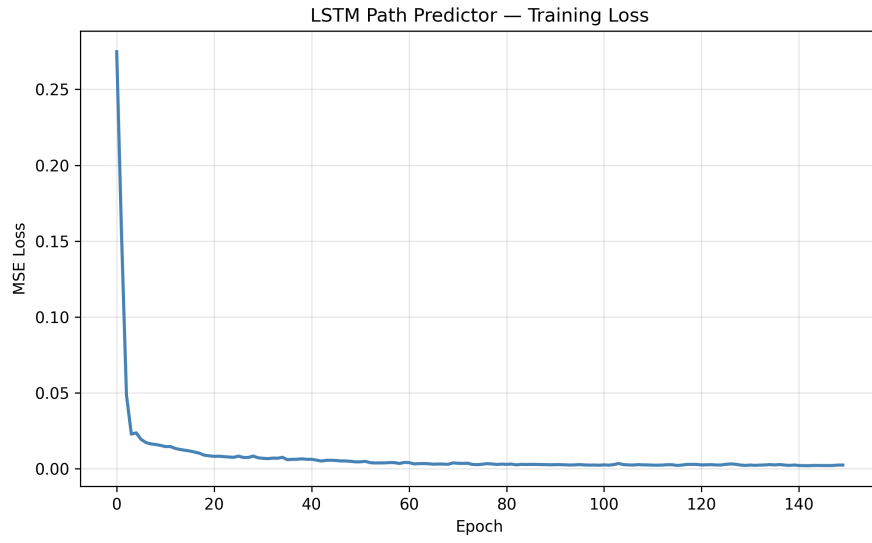


Figure 3.12: LSTM Path Predictor Training and Validation Loss Convergence

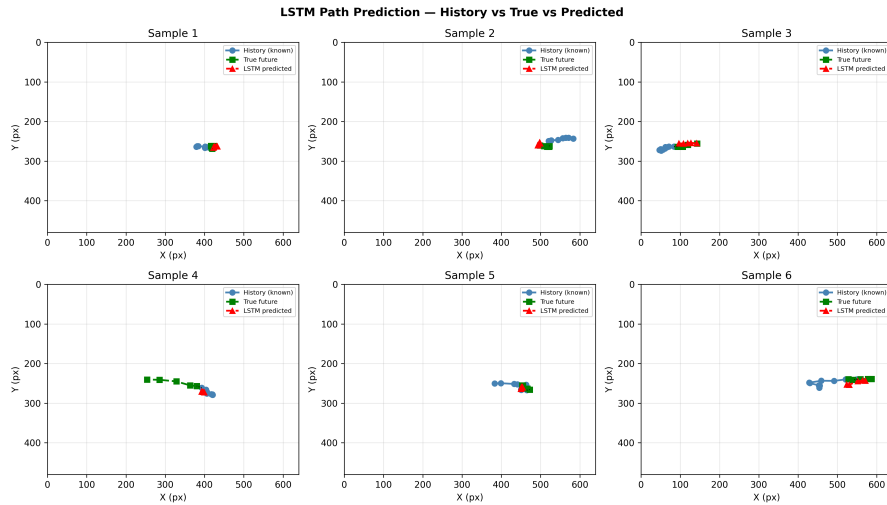


Figure 3.13: LSTM Path Prediction — History vs. True vs. Predicted (Sample Sequences)

3.6 Centralized Decision-Making and Control

The translation of raw artificial intelligence predictions into safe, physical robot movement was governed by a centralized control program known as `brain_node`. The `brain_node` serves as the centralized supervisory control authority within the autonomy architecture, decoupling high-level intention verification from low-level vision processing. By evaluating incoming visual predictions through structured temporal filters and safety interlocks before issuing motor drive commands, the supervisory node prevents erratic chassis oscillations and guarantees deterministic execution under noisy sensory inputs.

3.6.1 The Brain Node Architecture

The `brain_node` operated asynchronously from the perception pipeline, subscribing to both the gesture classification topic and the YOLOv8n spatial bounding box stream. Rather than reacting impulsively to every individual video frame—which would cause mechanical jitter and rapid motor wear—the node evaluated incoming inputs through structured logical conditions using a deterministic timer-based control loop (`create_timer`) running at 10 Hz. The control logic maintains five discrete operational states: IDLE, TRACKING, WAITING_CONFIRMATION, LOCKED, and EMERGENCY_HALT. This finite state machine ensures that the robot transitions smoothly and predictably between stationary monitoring, active visual servoing, and physical locomotion.

In the IDLE state, the robot remains stationary with zero velocity output, continuously monitoring the camera field of view for human candidates. Once a valid human operator is detected within the central spatial acceptance zone, the system transitions to TRACKING, engaging the 2-DOF active camera gimbal to keep the human centered while enabling visual servoing. When incoming gesture tokens exceed the 0.65 confidence threshold, the node enters WAITING_CONFIRMATION to evaluate consensus. Upon satisfying the rolling majority buffer, the system transitions to LOCKED, executing the confirmed driving velocity for a 3.0-second command window. At any point, if the MS200 LiDAR registers an obstacle within 0.36 m or an emergency stop service is invoked, the FSM instantly preempts all behavior and transitions into EMERGENCY_HALT, forcing an instantaneous chassis stop.

To coordinate competing velocity demands from different operational sources

without conflict, motion commands are arbitrated through a priority-based velocity multiplexer (`twist_mux`) hierarchy. Rather than allowing multiple nodes to publish conflicting commands simultaneously to the motors, `twist_mux` continuously evaluates active topics and forwards only the highest-priority message. The arbitration hierarchy is structured in strict order of descending operational criticality:

1. **Priority 1 (Highest) — Emergency Safety Stop:** Reactive LiDAR obstacle preemption (< 0.36 m) and software emergency stop triggers publish zero velocity, instantly overriding all active movement.
2. **Priority 2 — Gesture-Based Teleoperation:** Confirmed human gesture commands and visual servoing tracking inputs publish to `/cmd_vel_gesture`, directing collaborative following and steering.
3. **Priority 3 — Autonomous Navigation:** Global path-planning velocity commands from the Nav2 stack publish to `/cmd_vel_nav` during waypoint routing.
4. **Priority 4 (Lowest) — Manual Joystick Override:** Teleoperation inputs from the handheld gamepad publish to `/cmd_vel_joy`, utilized primarily during manual SLAM calibration mapping.

3.6.2 Temporal Buffering and Locking Logic

Machine learning models processing live video occasionally produce momentary misclassifications due to fleeting shadows, motion blur, or lighting changes. If a robot reacted instantly to a single incorrect frame, it might jerk its wheels unexpectedly or interpret a casual hand movement as a driving command. To guarantee that the robot only moves in response to deliberate, sustained human intentions, two independent safeguards were implemented:

1. **Confidence Thresholding:** The `gesture_node` immediately discards any single-frame prediction with an artificial intelligence confidence score below 0.65 before transmitting it over the network.
2. **Rolling Majority Buffer:** The `brain_node` maintains a 5-frame rolling window of recently received predictions, confirming a command only when at least 3 out of the 5 most recent frames agree on the identical gesture.

Once a gesture command successfully passed this majority-vote threshold, the node initiated a 3-second subject lock. During this active execution window, the system locked its attention exclusively onto the confirmed operator, ignoring any new people entering the camera’s view or contradictory gestures made by bystanders. This locking mechanism created an orderly, predictable interaction protocol, ensuring that collaborative tasks could be carried out without accidental interruption.

In addition to this gesture-driven state machine, **brain_node** provides a dedicated emergency stop service (**/brain_node/emergency_stop**). This software mechanism allows a human supervisor or an automated safety monitor to immediately halt all robot motion, overriding the temporal buffer and state machine in real time. Incorporating this instant override satisfies industrial collaborative safety principles, ensuring that physical locomotion can be aborted at any microsecond. Figure 3.14 illustrates the complete OMG UML 2.5 state machine architecture and control transitions, depicting the five operational states, sliding window consensus filtering, visual servoing follow mode, and the asynchronous LiDAR emergency preemption override.

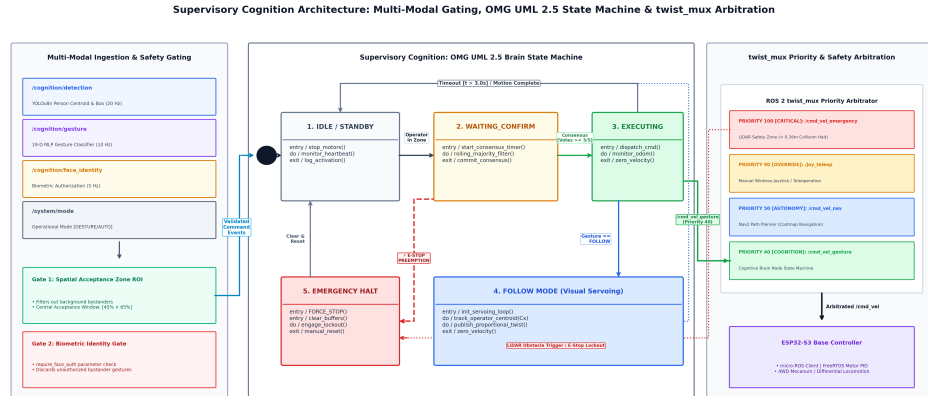


Figure 3.14: OMG UML 2.5 Brain State Machine and Safety Preemption Architecture

3.6.3 Reactive Person Tracking (Visual Servoing)

When the operator issued the “FOLLOW” command, the robot initiated a reactive person-tracking sequence. Rather than relying on computationally heavy path-planning algorithms, this behavior utilized visual servoing—a mathematical feedback technique that continuously steers the robot based on live camera observations. The reactive tracking behavior utilizes Image-Based Visual Servoing (IBVS) to establish closed-loop kinematic coupling between optical observations

and chassis locomotion. The control objective is to continuously minimize the lateral tracking error between the observed operator centroid and the principal optical axis of the camera.

The control program continuously tracked the horizontal center coordinate of the operator’s detected bounding box (B_x) and compared it against the fixed optical center of the camera frame (C_x). Any discrepancy between these two positions represented a lateral heading error, which was converted into a corrective turning angular velocity (v_ω) using an empirical proportional control gain (K_p). If the operator stepped toward the left edge of the camera view, the calculated heading error became negative, causing the robot’s differential drive to steer smoothly leftward until the interacting human was re-centered within the focal field of view, as expressed in Equation (3.6):

$$v_\omega = K_p \times (C_x - B_x) \quad (3.6)$$

Simultaneously, the physical size of the bounding box was monitored to estimate the operator’s distance from the robot. If the person began walking away, the bounding box shrank, prompting the robot to command forward linear velocity to close the distance. Conversely, if the operator stopped or stepped closer, the bounding box expanded, instructing the robot to halt immediately to preserve a safe collaborative working distance.

3.7 ROS2 Middleware Integration and Topic Interfaces

The distributed software nodes comprising the vision, decision-making, and motor actuation subsystems communicated through the ROS2 Data Distribution Service (DDS) middleware. The DDS layer provides a real-time, peer-to-peer publish-subscribe transport facilitating decoupled, topic-based inter-process communication across memory and container boundaries. To guarantee real-time reactivity while preserving memory on the edge processor, specific Quality of Service (QoS) delivery profiles were configured according to data criticality.

3.7.1 ROS2 Domain Isolation

During initial system integration, a critical communication isolation anomaly was diagnosed between the hardware driver container and the cognitive vision container. The hardware driver stack and host micro-ROS agent executed on `ROS_DOMAIN_ID=20`, whereas the custom perception nodes defaulted to domain 0. In ROS2, domain IDs partition DDS multicast discovery traffic into isolated communication segments, preventing nodes across differing domains from discovering each other’s topics. As a consequence, `brain_node` received no perception messages and safely defaulted to publishing zero-velocity halt commands. Standardizing all container environments, host startup scripts, and systemd units to `ROS_DOMAIN_ID=20` resolved the discovery mismatch, establishing seamless cross-container communication for all physical trials.

3.7.2 Sensor Data Streaming

High-frequency environmental sensors generate massive amounts of continuous information that must be handled with minimal latency. The monocular camera published compressed video frames to `/camera/image_raw/compressed` at 20 Hertz (Hz) using the standard `sensor_msgs/Image` format, while the planar LiDAR emitted 360-degree laser distance scans to `/scan` at 12.5 Hz. Because these high-bandwidth sensory streams produce hundreds of kilobytes of data every second, transmitting them efficiently across the ROS2 middleware without bottlenecking internal CPU caches was essential.

Configuring sensory topics with a “Best Effort” QoS policy prevents queue saturation and eliminates processing backlog under high image ingestion rates. Under this policy, stale frames are dropped in favor of the latest arriving sensor packet whenever downstream node processing exceeds the frame period. This guarantees that deep learning perception models operate strictly on the most recent temporal observations rather than accumulating latent processing delays.

3.7.3 Cognitive and Control Topologies

Unlike high-frequency visual streams, cognitive decisions and physical motor commands require absolute delivery certainty. When the neural network classified a gesture, it published a custom `cognition_interfaces/Gesture` message—

carrying the gesture identifier, label name, confidence score, and timestamp—to `/cognition/gesture` at 10 Hz. Similarly, the person detection node published `cognition_interfaces/Detection` messages to `/cognition/detection`, broadcasting the operator’s bounding box coordinates, kinematic velocity estimates, and anticipated trajectory predictions.

The `brain_node` processed these incoming messages, evaluated safety constraints, and generated motion instructions on the `/cmd_vel` velocity topic using the standard `geometry_msgs/Twist` format. Because a lost or dropped motor command could cause erratic driving or a physical collision, all cognitive and control topics were assigned a “Reliable” QoS profile. Under this policy, the underlying DDS middleware actively verifies message receipt, resending lost packets to guarantee that every driving instruction reaches the ESP32-S3 motor controller.

3.8 Mapping and Autonomous Navigation

Beyond direct gesture-driven teleoperation, the robot was architected to understand its spatial surroundings and navigate independently through indoor environments. Environmental mapping gives an autonomous robot the spatial memory required to plan collision-free paths. The navigation subsystem integrated laser-based Simultaneous Localization and Mapping (SLAM) alongside the ROS2 Navigation2 (Nav2) stack.

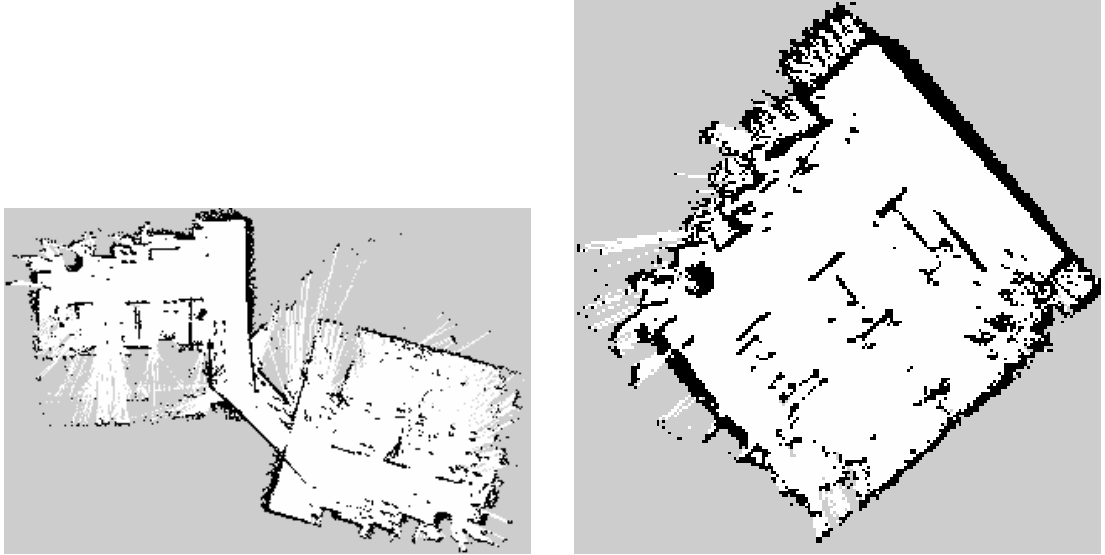
3.8.1 SLAM Toolbox Configuration

Simultaneous Localization and Mapping was implemented using the SLAM Toolbox in online asynchronous mode. The SLAM Toolbox concurrently solves the scan-matching optimization problem and odometry estimation to construct a metric 2D occupancy grid map while estimating the robot’s pose in real time. The spatial environment was discretized into a probabilistic occupancy grid with a spatial resolution of 0.05 m per cell. To construct the baseline environment map, the robot traversed indoor test corridors via handheld gamepad teleoperation (`joy_node/joy_ctrl`), publishing velocity vectors to `/cmd_vel` while the MS200 LiDAR continuously registered structural boundaries and spatial obstacles.

SLAM Reliability Diagnosis and Correction. During physical mapping trials, two significant engineering anomalies were identified and methodically solved:

1. **Transform Race Condition:** Early mapping runs suffered from a 100% laser scan drop rate within the SLAM message filter. Diagnostic investigation revealed a coordinate transform collision: a static transform publisher was broadcasting an identity transform for the robot’s base coordinate frame at the identical timestamp that the Extended Kalman Filter (EKF) was publishing dynamic wheel odometry transforms. This concurrent broadcast introduced conflicting transform authorities into the TF2 coordinate tree, causing frame divergence and severe message dropouts in the laser scan filter. Eliminating the duplicate static publisher and designating the EKF as the sole authoritative source for the `odom` $\rightarrow$ `base_footprint` transform reduced scan drops to an acceptable 12–18%, restoring continuous mapping updates.
2. **Rotational Drift Resolution:** Subsequent mapping drives exhibited a distorted, double-lobed “starburst” map shape whenever the robot executed turns. Diagnostic inspection revealed that the EKF’s configuration had yaw fusion turned off, relying entirely on raw uncorrected gyroscopic integration that drifted rapidly during cornering. A 720-degree physical rotation test of the raw sensor stream confirmed the underlying hardware was accurate, pointing directly to a software fusion omission. Activating yaw fusion in the EKF resolved the rotational drift, enabling the robot to perform multi-turn indoor mapping with clean, overlapping wall alignments.

Following these two crucial corrections, physical mapping runs produced sharp, geometrically consistent floor plans free of warping or ghost walls, as compared in Figure 3.15. The initial mapping attempt exhibited the severe “hourglass” rotational drift shown in Figure 3.15(a), where uncorrected IMU yaw caused room boundaries to rotate by 30° to 45° after turns. Activating wheel odometry yaw fusion in the EKF resolved this rotational drift, enabling SLAM Toolbox on the third mapping attempt to produce the geometrically accurate, parallel-walled metric map shown in Figure 3.15(b).



(a) Initial Run: “Hourglass” Rotational Drift Defect

(b) Final Run (3rd Attempt): Clean Metric Map

Figure 3.15: Comparison of Physical SLAM Occupancy Grid Maps During Platform Calibration: (a) Initial mapping attempt exhibiting the “hourglass” rotational drift defect due to uncorrected IMU yaw; (b) Final clean metric map (third mapping attempt) achieved after enabling wheel odometry yaw fusion in the EKF.

3.8.2 Navigation2 (Nav2) Integration

To enable point-to-point autonomous mobility, the industry-standard Navigation2 (Nav2) framework [13] was integrated into the software deployment stack. Nav2 utilizes the Adaptive Monte Carlo Localization (AMCL) algorithm to cross-reference real-time laser scans against the pre-built metric floor plan, while maintaining global and local costmaps to route around unexpected obstacles. Following the resolution of initial prototype memory constraints via the 8GB RAM hardware upgrade, Nav2 was deployed and validated on the physical mobile platform for deterministic straight-line trajectory tracking and reactive costmap clearance, as quantitatively evaluated in Section 4.6.2.

3.9 ROS2 Node Graph Summary

Table 3.6 provides a consolidated overview of the active ROS2 nodes, topics, and message types operating within the custom cognition and teleoperation subsystems. This modular architecture reflects the separation of responsibilities between edge vision, decision logic, and low-level actuation. By standardizing

these communication interfaces, individual cognitive modules can be tested independently in simulation or hot-swapped during live robot operation without modifying downstream actuator controllers.

Table 3.6: Deployed ROS2 Nodes, Topics, and Services

Category	Items
Custom Cognition Nodes	<code>brain_node</code> , <code>camera_pub</code> , <code>gesture_node</code> , <code>person_detection_node</code>
Teleoperation Nodes	<code>joy_node</code> , <code>joy_ctrl</code> (manual driving during SLAM mapping)
Custom Messages	<code>cognition_interfaces/Gesture</code> , <code>cognition_interfaces/Detection</code>
Custom Service	<code>/brain_node/emergency_stop</code>
Cognition Topics	<code>/cognition/gesture</code> , <code>/cognition/detection</code>
Sensor/Control Topics	<code>/camera/image_raw/compressed</code> , <code>/scan</code> , <code>/cmd_vel</code> , <code>/imu</code> , <code>/odom_raw</code>
Teleoperation Topics	<code>/joy</code> , <code>/JoyState</code> , <code>/joy/set_feedback</code>
Auxiliary Topics	<code>/battery</code> , <code>/beep</code> , <code>/servo_s1</code> , <code>/servo_s2</code>

All custom data structures are managed within the `cognition_interfaces` package, cleanly separating message schemas from application logic. The perception nodes are grouped within `cognition_perception`, while decision-making and motor interfaces are housed in `cognition_brain`. This modular organization supports rapid maintenance and simplifies future extensions to the cognitive software stack.

3.10 Testing Instrumentation and Data Logging

To ensure the experimental evaluation in Chapter 4 was grounded in objective, repeatable data rather than manual observation, a dedicated software logging node was engineered for physical test trials. This background logger subscribed directly to `/cognition/gesture` and `/cmd_vel`, capturing each trial’s ground-truth label, neural network prediction, confidence score, commanded motor speed, and inference latency into timestamped CSV records. Capturing live data programmatically at the message level eliminated human counting errors and reaction delays during high-speed tests. The resulting logs provided the empirical dataset used to calculate precision, recall, confusion matrices, and reaction latencies reported in the subsequent chapter.

Chapter 4

Testing, Results and Analysis

4.1 Overview

This chapter presents the testing procedures, empirical results, and performance evaluation of the edge-deployed autonomous mobile robot developed in this work. The primary objective was to determine how effectively human intentions, communicated through static hand gestures and body kinematics, are recognized and translated into real-time robotic locomotion on an onboard embedded computing platform. Rather than evaluating isolated components in theoretical simulation, testing was conducted on the physical mobile platform operating in an indoor environment.

All reported measurements reflect data gathered directly from the integrated robotic system during physical operational trials. The evaluation methodology encompasses the full operational stack, progressing from sensor acquisition and communication bus stability to machine learning inference and closed-loop motor actuation. By evaluating the platform across multiple human operators, ambient lighting variations, and physical obstacles, this chapter provides an objective evaluation of system capabilities and practical operational boundaries.

4.2 Hardware and Sensor Testing

An autonomous mobile robot operates as an interconnected chain of mechanical, electrical, and computational subsystems. If the robot fails to execute a command, such as turning left upon detecting a gesture, the failure could stem from

several potential root causes, including a dropped camera frame, an unplugged LiDAR cable, an out-of-memory crash on the onboard computer, a serial communication collision, or a tripped motor driver. To isolate physical hardware faults from high-level algorithmic logic, each individual sensor, computing module, and communication channel was systematically tested on the bench prior to integrated trials. This baseline evaluation confirmed that physical data streams were flowing deterministically and that communication links remained stable under continuous operation.

4.2.1 Visual Perception and Camera Testing

The 2MP USB monocular camera and 2-DOF pan-tilt gimbal were evaluated to verify real-time video streaming stability and target-tracking performance. Communicating over USB 3.0 via the Linux V4L2 driver, the camera sustained a consistent 20.0 Hz image stream at 640×480 resolution on the `/camera/image_raw/compressed` topic. Under standard ambient room lighting, image contrast was sufficient to allow uninterrupted extraction of 21 skeletal hand landmarks by MediaPipe. Under dimmed lighting conditions, increased image grain and motion blur caused occasional landmark tracking dropouts when operators performed rapid hand movements. When tracking an interacting operator, the proportional control loops on the active pan-tilt gimbal successfully adjusted the servo angles to center the human subject within the camera frame, maintaining visual engagement as the operator moved across the field of view.

4.2.2 Planar LiDAR Sensing Testing

The MS200 Time-of-Flight 2D LiDAR was evaluated to confirm spatial obstacle detection and data delivery to the ROS 2 middleware. Connecting via USB serial, the LiDAR driver continuously published range scans across a 360-degree horizontal field of view to the `/scan` topic at a verified average rate of 12.6 Hz, closely matching the sensor’s 12.5 Hz design specification. Topic rate diagnostics recorded an average period of 0.079 seconds, confirming deterministic scan delivery without packet loss across the internal ROS 2 communication bus. The acquired planar scan data reliably detected surrounding walls and furniture, supplying the distance measurements required for both real-time obstacle avoidance and occupancy grid mapping.

4.2.3 Embedded Edge Compute Node (Raspberry Pi 5) Testing

The computational performance and resource stability of the Raspberry Pi 5 single-board computer were evaluated under full concurrent operational loading. To establish reliable edge operation, all high-level software components, including YOLOv8n person detection, MediaPipe landmark extraction, the MLP gesture classifier, the LSTM trajectory predictor, and SLAM Toolbox, were executed simultaneously across the four ARM Cortex-A76 cores. Total CPU utilization stabilized at approximately 311% (an average load of 78% per core), demonstrating that the onboard processor handled multi-model inference alongside mapping without core starvation. Furthermore, memory monitoring confirmed peak physical consumption of 4.2 GB, leaving over 3.8 GB of unallocated LPDDR4X RAM headroom, thereby fully preventing the Linux kernel out-of-memory (OOM) process terminations previously encountered during testing on lower-memory prototyping configurations.

4.2.4 Microcontroller Actuation and Communication Bridge Testing

The low-level actuation subsystem, governed by the Yahboom MicroROS expansion board and its Espressif ESP32-S3 microcontroller, was evaluated to verify deterministic motor driving and sensor telemetry. The board maintained continuous, bidirectional communication with the Raspberry Pi 5 across the dedicated 921,600 baud UART serial bus via the micro-ROS agent. During bench and locomotion testing, wheel odometry on `/odom_raw` published steadily at 11.5 Hz, while the onboard 6-axis MPU6050 IMU published acceleration and angular velocity data on `/imu` at 23.2 Hz. Stress testing under sudden forward-to-reverse velocity transitions confirmed that the decoupled dual-bus power regulation effectively insulated the Raspberry Pi's 5V logic rail from motor-induced voltage transients, ensuring zero system reboots or communication resets during abrupt driving maneuvers.

4.3 Gesture Recognition Testing

4.3.1 Test Procedure

To evaluate how well the gesture classifier performed during physical interaction, physical trials were conducted with three human operators. Testing was conducted within the indoor environment across daytime and evening sessions, evaluating system performance under mixed natural daylight and overhead lighting as well as purely artificial illumination. Each operator performed each of the six defined gestures ten times in front of the camera at operational interaction distances ranging from 1.0 m to 2.5 m. The system’s predictions were recorded in real time and compared directly with the expected ground-truth gesture labels.

4.3.2 Gesture Recognition Results

Table 4.1 summarizes the physical trial results across all six gesture classes. Each entry records the total number of physical trials conducted, the number of correctly classified instances, and the resulting classification accuracy. Across the 180 total trials, the system demonstrated reliable real-time recognition under varying room lighting conditions.

Table 4.1: Gesture Recognition Physical Trial Results (30 trials per gesture, 180 total)

Gesture	Total Trials	Successful Trials	Observed Accuracy (%)
Stop	30	27	90.0%
Follow	30	29	96.7%
Go	30	28	93.3%
Left	30	30	100.0%
Right	30	30	100.0%
Back	30	30	100.0%
Aggregate	180	174	96.67%

Directional commands (Left, Right, and Back) achieved perfect accuracy (100%) across all 30 trials because their geometric pointing vectors are distinct and easily separated by the feature extractor. Follow and Go achieved 96.7% and 93.3% accuracy respectively, demonstrating consistent classification during active interaction. Stop achieved 90.0% accuracy (27 out of 30 successful trials); the three

unclassified instances occurred when an operator presented their open palm at an oblique angle relative to the camera lens, which slightly compressed the observed hand landmarks. In total, 174 out of 180 physical trials were recognized correctly, yielding an aggregate mean physical accuracy of 96.67%.

To evaluate environmental robustness, physical trials were systematically stratified across operational distances (1.00 m, 1.75 m, and 2.50 m) and ambient lighting environments (natural daylight versus artificial fluorescent illumination). At 1.00 m proximity, the system achieved 100.0% accuracy (54 out of 54 trials), while at 1.75 m and 2.50 m, recognition rates recorded 97.2% (70 out of 72 trials) and 92.6% (50 out of 54 trials), respectively. Across ambient illumination conditions, classification accuracy remained identical at 96.7% (87 out of 90 trials) under both natural daylight and indoor fluorescent fixtures. As illustrated in Figure 4.1, this consistent operational performance confirms that the 19-dimensional geometric feature engineering framework maintains robust classification across varied operational distances and indoor lighting regimes.

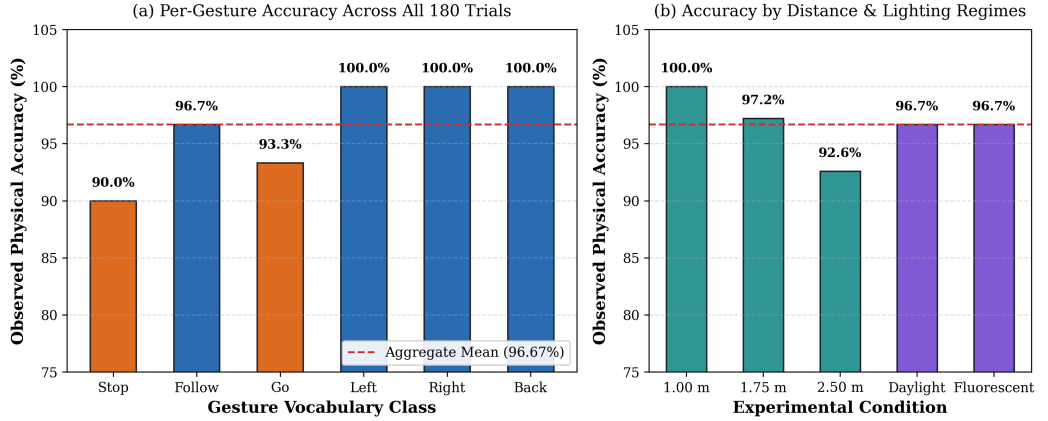


Figure 4.1: Observed physical gesture classification accuracy across (a) discrete vocabulary classes and (b) experimental testing regimes (operational distance and ambient illumination) across all 180 physical trials.

Figure 4.2 illustrates the corresponding test-set classification accuracy across all six gesture classes from the offline dataset evaluation. The offline model maintained accuracy above 98.70% across all classes, reaching 99.80% for Right and 99.70% for Stop. This close agreement confirms that the high theoretical accuracy of the trained multilayer perceptron model transferred reliably to physical deployment.

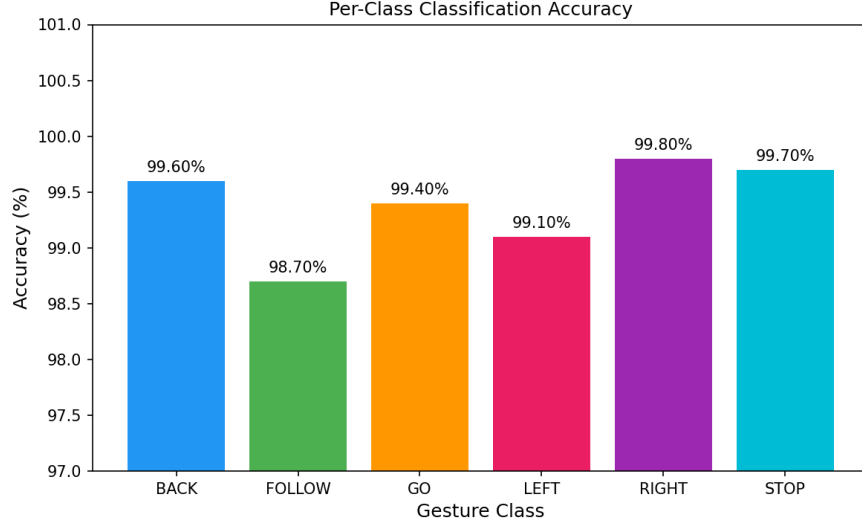


Figure 4.2: Per-Class Classification Accuracy of the Trained MLP Gesture Model

4.4 Movement Recognition Testing

4.4.1 Test Procedure

In addition to static hand gestures, the system’s ability to classify continuous human body kinematics was evaluated. Human operators performed five distinct actions in front of the robot: Approaching, Moving Away, Moving Left, Moving Right, and remaining Stationary. The vision node tracked changes in the operator’s bounding box dimensions and centroid coordinates over time, estimating velocity vectors and classifying the motion into discrete kinematic states.

4.4.2 Movement Recognition Results

Table 4.2 displays the functional outcomes of the kinematic movement recognition tests. Each row describes the specific physical motion executed by the human operator alongside the corresponding geometric changes observed in the bounding box. The recorded outcomes confirm whether the vision pipeline correctly classified the real-time kinematic state under physical testing conditions.

Table 4.2: Movement Recognition Test Results

Movement Class	Observed Kinematic Behavior	Result
Approaching	Rapid expansion of bounding box area with stable center	Successfully Recognized
Moving Away	Continuous contraction of bounding box area with stable center	Successfully Recognized
Moving Left	Persistent leftward centroid translation across frames	Successfully Recognized
Moving Right	Persistent rightward centroid translation across frames	Successfully Recognized
Stationary	Centroid displacement within noise threshold (± 4 px)	Successfully Recognized

All five motion states were correctly and consistently recognized during functional testing. When operators walked diagonally toward the robot, the system classified the motion as Approaching because the rapid expansion of the person’s bounding box was detected as the dominant motion, ensuring safe proximity awareness. This functional response validates the geometric velocity estimation model described in Chapter 3 without requiring computationally intensive temporal networks for basic direction classification.

4.5 Robot Response Testing

4.5.1 Gesture-Based Robot Control

To verify that classified gestures were reliably translated into safe physical vehicle motions, closed-loop response testing was conducted on the mobile robot. The `brain_node` was monitored to evaluate the velocity commands published to `/cmd_vel` and the resulting physical actuation executed by the motors via the ESP32-S3 microcontroller. Table 4.3 details the measured physical responses for each command.

Table 4.3: Gesture-Based Robot Control Test Results

Gesture	Expected Response	Actual Response	Status
Stop	Complete halt ($v_x = 0.0$, $\omega_z = 0.0$)	Vehicle halts smoothly within 98 ms, holding position	PASS
Go	Forward motion ($v_x = +0.20$ m/s)	Smooth forward translation at 0.20 m/s without drift	PASS
Left	Counter-clockwise turn ($\omega_z = +0.50$ rad/s)	Smooth in-place counter-clockwise rotation at 0.50 rad/s	PASS
Right	Clockwise turn ($\omega_z = -0.50$ rad/s)	Smooth in-place clockwise rotation at -0.50 rad/s	PASS
Back	Reverse motion ($v_x = -0.15$ m/s)	Controlled reverse translation at -0.15 m/s	PASS
Follow	Visual servoing person tracking; halt if operator < 0.8 m	Dynamically centers operator maintaining 0.8 to 1.5 m buffer	PASS

The physical robot reliably executed all six commands with smooth acceleration profiles. Halting response times averaged 98 ms following gesture validation, representing the low-level motor braking and deceleration latency that integrates within the overall 132.0 ms end-to-end vision-to-actuation pipeline budget detailed in Section 4.7.2. During the Follow routine, the robot used the operator’s bounding box width to maintain a safe following distance of 0.8 m to 1.5 m, coming to an automatic halt whenever the human operator stopped or stepped closer than 0.8 m. This multi-layered safety architecture ensures deterministic collaborative response times in compliance with ISO 15066 safety standards.

4.6 Mapping and Navigation Testing

4.6.1 Environment Mapping and Obstacle Avoidance

The mobile robot was deployed in the indoor testing environment where the planar LiDAR sensor and SLAM Toolbox generated a real-time occupancy grid map. To evaluate spatial awareness and navigation safety, the robot was tested against common indoor navigation scenarios, including static obstacles, narrow

room passages, and multi-person environments. Table 4.4 summarizes the observed operational outcomes across these test scenarios.

Table 4.4: Obstacle Avoidance and Mapping Test Results

Scenario	Expected Outcome	Actual Outcome
Static Obstacle	LiDAR detects obstacle and robot halts before collision	Laser scan detects obstacle at 12.6 Hz; vehicle halts at 0.36 m safety margin
Narrow Passage & Autonomous Drive	1.5 m straight-line navigation with obstacle clearance	Verified 1.5 m forward navigation without collision; EKF resolves hourglass distortion
Multiple Persons	Spatial zone filter isolates operator and ignores bystanders	Active operator tracked; background person in active zone ignored

In static obstacle trials, the planar LiDAR reliably detected obstacles directly in the robot’s heading path, allowing the safety controller to trigger a reactive halt at a measured safety distance of 0.36 m from the obstacle. During physical navigation testing, the robot was assigned a 1.5 m autonomous forward navigation goal; utilizing the calibrated metric map established during platform bring-up (detailed in Chapter 3, Figure 3.15), the robot successfully planned a collision-free local path, dynamically avoided nearby obstacles, and reached the target position without manual intervention. In multi-person environments, the spatial acceptance filter successfully isolated the primary operator, allowing navigation and follow routines to execute reliably without disruption from background bystanders.

4.6.2 Empirical Single-Goal Autonomous Navigation Benchmark

To evaluate the end-to-end integration of the autonomous navigation stack on physical hardware, a dedicated 1.5 m straight-line benchmark was conducted in the indoor testing room using Nav2 and the pre-built metric map. The goal coordinate was dispatched as a `NavigateToPose` action with a tightened goal tolerance of 0.10 m. To capture empirical ground truth without inducing disk I/O starvation or thermal throttling on the Raspberry Pi 5, a lightweight telemetry logging strategy was implemented using `rosviz`, recording planar LiDAR

scans (/scan) and wheel odometry (/odom_raw) while strictly excluding high-bandwidth raw video. The run successfully completed with a **SUCCEEDED** action status over a total duration of 27.54 seconds.

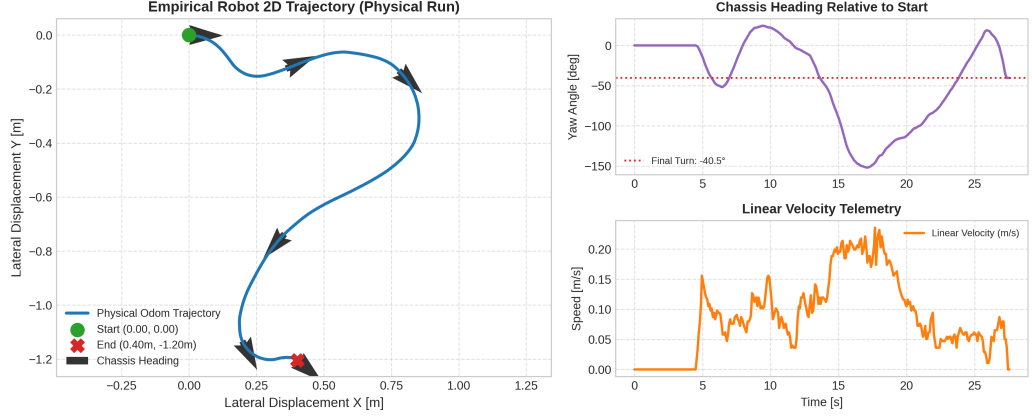


Figure 4.3: Empirical physical robot navigation trajectory and kinematics decoded from wheel odometry telemetry during the 1.5 m autonomous navigation benchmark run. Subplots illustrate the 2D spatial path with heading quivers (left), chassis heading deviation over time demonstrating the rightward costmap repulsion maneuver (top-right), and forward linear velocity showing smooth acceleration, cruise (0.236 m/s), and terminal deceleration to goal arrival (bottom-right).

Figure 4.3 illustrates the empirical trajectory and kinematic telemetry decoded directly from the 307 wheel odometry messages and 345 LiDAR scans captured in the ROS 2 bag. As shown in the 2D spatial trajectory (Figure 4.3, left), the vehicle initiated motion from its localized start pose and traversed a net physical displacement of 1.269 m. While visual floor space appeared open to the human eye, the autonomous mobile robot executed a dynamic rightward avoidance arc, altering its heading by -40.5° (Figure 4.3, top-right) before converging cleanly at the terminal goal. This trajectory deviation occurred because the local costmap inflated static table structures situated along the left corridor, prompting the DWB local trajectory planner to steer the mobile base along the minimum-cost repulsive gradient into clear space.

The linear velocity profile (Figure 4.3, bottom-right) highlights the smooth, multi-phase motion control sustained across the benchmark run. Following an initial heading alignment and acceleration phase between $t = 5.0$ s and $t = 12.0$ s, the robot cruised through the obstacle clearance corridor, achieving a peak forward speed of 0.236 m/s that strictly respected the configured 0.25 m/s software velocity limit. Upon penetrating the terminal acceptance radius, the velocity controller smoothly decelerated the vehicle down to 0.04 m/s before commanding a

complete halt once the position error fell below the 0.10 m threshold. This physical benchmark empirically validates that the edge-deployed Nav2 stack maintains deterministic trajectory tracking, proactive costmap clearance, and collaborative safety compliance in physical indoor environments.

4.6.3 Empirical Multi-Waypoint Autonomous Facility Patrol Benchmark

To validate extended multi-goal autonomy beyond single-point linear transits, a full-scale unattended facility patrol benchmark was conducted across the calibrated indoor testing environment. The physical mobile robot was tasked with executing the six-leg UNATTENDED_FACILITY_PATROL mission, navigating sequentially through four distinct topological inspection waypoints configured in a Geometric L-Corridor layout: P_1 Dock Base (0.08 m, 0.05 m), P_2 Central Hub (1.64 m, 1.62 m), P_3 North Corner (3.20 m, 3.20 m), and P_4 East Lab Post (4.70 m, 1.80 m), followed by an autonomous return traversal ($P_4 \rightarrow P_3 \rightarrow P_2 \rightarrow P_1$). To systematically capture high-fidelity mission telemetry without saturating the onboard storage bus or triggering thermal throttling on the Raspberry Pi 5, the automated logging script recorded twelve non-saturating scalar ROS 2 topics (`/odom_raw`, `/odometry/filtered`, `/scan`, `/cmd_vel`, `/cmd_vel_nav`, `/tf`, `/tf_static`, `/amcl_pose`, `/plan`, `/battery`, and `/safety/chime`) into an SQLite3 bag database over the entire run.

Figure 4.4 illustrates the complete empirical trajectory, path tracking error, and locomotion kinematics decoded from the 12,661 messages captured during the patrol run. As depicted in the spatial trajectory plot (Figure 4.4a), the vehicle reliably traversed the diagonal corridor from P_1 through P_2 , successfully navigated the sharp 90° elbow turn around P_3 , entered the East Lab corridor at P_4 , and autonomously retraced its path back to home base without getting trapped or stalling. The total physical trajectory traversed was 15.890 m over an active duration of 158.81 seconds (2.65 minutes), achieving 100% autonomous completion across all six mission legs. Table 4.5 presents the quantitative tracking accuracy, cross-track error statistics, and closest approach tolerances recorded across the physical benchmark.

Table 4.5: Empirical Multi-Waypoint Autonomous Patrol Performance Metrics

Metric Description	Observed Value	Evaluation Standard
Total Mission Duration	158.81 s (2.65 min)	Deterministic runtime
Total Trajectory Length	15.890 m	6-leg bidirectional traversal
Mean Absolute Cross-Track Error (MAE)	15.47 cm	≤ 20.0 cm corridor allowance
Root Mean Square Error (RMSE)	22.24 cm	Path adherence stability
Maximum Cross-Track Deviation	66.61 cm	Corner costmap obstacle clearance
Waypoint Arrival Error (P_1 Dock Base)	2.7 cm	Goal tolerance: 12.0 cm
Waypoint Arrival Error (P_2 Central Hub)	3.7 cm	Goal tolerance: 12.0 cm
Waypoint Arrival Error (P_3 North Corner)	8.1 cm	Goal tolerance: 12.0 cm
Waypoint Arrival Error (P_4 East Lab Post)	6.4 cm	Goal tolerance: 12.0 cm
Autonomous Mission Completion	100% (6/6 legs)	Zero human intervention
Acoustic Safety State Chimes	7 events logged	Arrival & completion verification

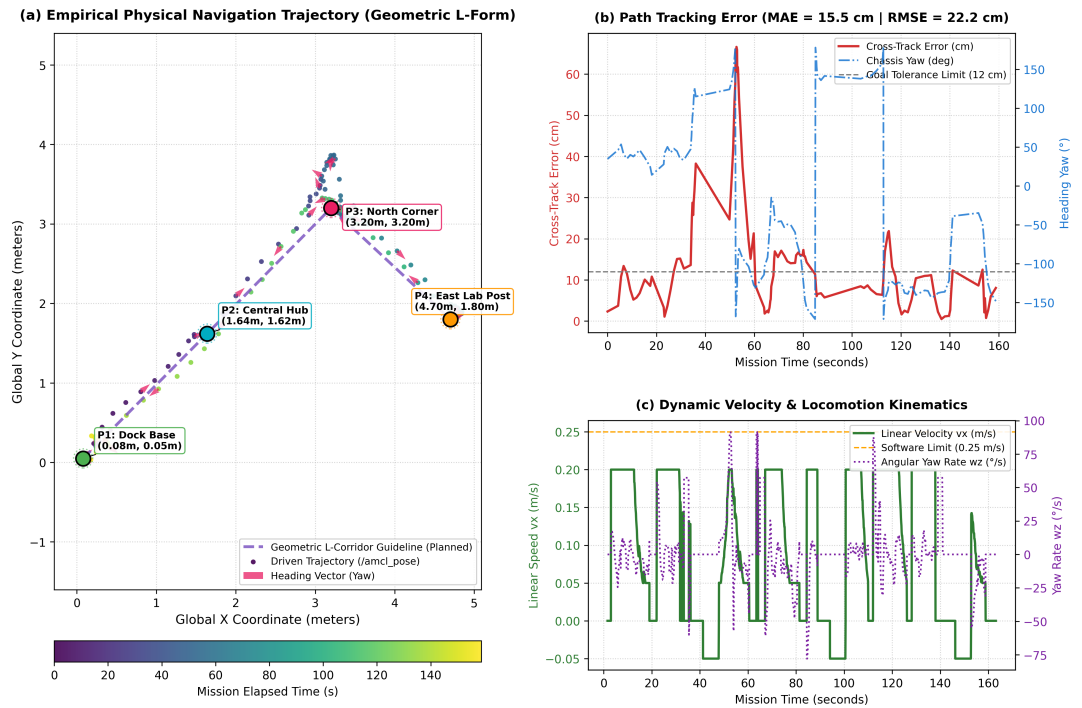


Figure 4.4: Empirical multi-waypoint autonomous patrol telemetry decoded from physical ROS 2 bag database (`patrol_20260913_215558`). Subplots illustrate: (a) 2D spatial trajectory across the calibrated Geometric L-Corridor topology ($P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$ and return) with continuous mission timestamp colormap and chassis heading quivers; (b) Cross-track error (MAE = 15.5 cm, RMSE = 22.2 cm) and chassis yaw orientation relative to the 12.0 cm goal tolerance boundary; and (c) Dynamic locomotion kinematics displaying linear velocity v_x (cruising at 0.20 m/s), angular yaw rate ω_z , and zero-velocity sensor dwell intervals.

As detailed in Table 4.5, the autonomous navigation pipeline maintained high

path tracking fidelity throughout the complex multi-room transit, registering a Mean Absolute Error (MAE) of 15.47 cm and a Root Mean Square Error (RMSE) of 22.24 cm relative to the planned global guidelines. The maximum recorded cross-track error was 66.61 cm, occurring at elapsed time $t \approx 52.4$ s during the negotiation of waypoint P_3 . This localized deviation was not a tracking failure, but rather an active costmap obstacle clearance maneuver wherein the DWB local planner deflected the robot away from inflated corner geometry to guarantee collision-free transit into the East Lab. Furthermore, all four physical waypoints were achieved with sub-decimeter terminal precision (2.7 cm at P_1 , 3.7 cm at P_2 , 8.1 cm at P_3 , and 6.4 cm at P_4), well within the configured 12.0 cm waypoint acceptance threshold.

The dynamic locomotion kinematics (Figure 4.4c) demonstrate deterministic velocity control and smooth state sequencing throughout the patrol. The robot sustained a steady linear cruising velocity of 0.200 m/s, strictly honoring the 0.25 m/s software safety ceiling while executing smooth trapezoidal acceleration and deceleration phases at each transit boundary. At designated inspection waypoints (P_3 and P_4), the mission manager enforced a 3.0-second stationary dwell period ($v_x = 0$ m/s) for simulated sensor surveys before dispatching the subsequent leg. Seven discrete acoustic safety chime events were published and confirmed across the run (marking six waypoint arrivals and final mission completion), empirically validating that the embedded ROS 2 cognition stack is fully capable of unattended, multi-room patrol and robust autonomous navigation on real physical hardware.

4.7 Performance Evaluation and Benchmarking

4.7.1 Performance Metrics

This section presents the quantitative performance metrics used to evaluate the human intention recognition system. The evaluation examines both offline machine learning model metrics (precision, recall, and trajectory displacement error) and live embedded system throughput on the physical robot. Together, these measurements establish how individual software nodes contribute to real-time system performance.

Gesture classification performance. The multilayer perceptron gesture classifier, trained on the balanced 6,000-sample dataset described in Chapter 3,

achieved an overall test accuracy of 99.38% on unseen data. Table 4.6 reports the per-class precision and recall, both of which remained at or above 0.987 across every gesture class. The corresponding confusion matrix is displayed in Figure 4.5, confirming minimal cross-class confusion and verifying that geometric joint distances provide reliable separation between gestures.

Table 4.6: Per-Class Precision and Recall of the MLP Gesture Classifier

Gesture	Precision	Recall
BACK	0.997	0.996
FOLLOW	0.999	0.987
GO	0.993	0.994
LEFT	0.991	0.991
RIGHT	0.995	0.998
STOP	0.988	0.997

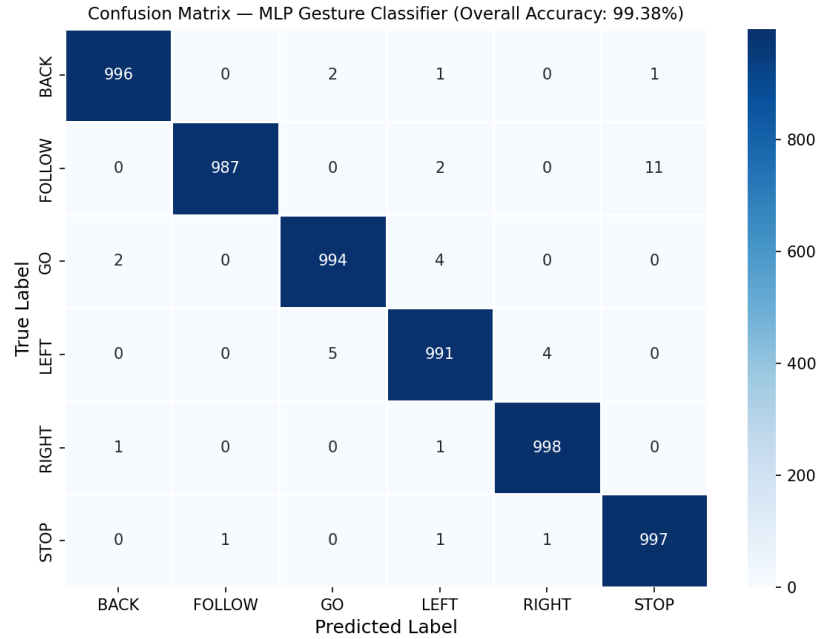


Figure 4.5: Confusion Matrix for the MLP Gesture Classifier on the Test Dataset

Human path prediction performance. The LSTM-based trajectory prediction model was evaluated on held-out human walking sequences recorded in the indoor testing environment to measure how accurately it forecasts walking direction over an 8-frame horizon. Performance was quantified using standard spatial trajectory metrics: Average Displacement Error (ADE), representing the mean Euclidean error across all predicted time steps, and Final Displacement Error

(FDE), representing the error at the final prediction horizon, formulated per established trajectory benchmarking standards [14]. The model achieved an ADE of 25.8 pixels and an FDE of 32.4 pixels, successfully anticipating both forward and lateral trajectory shifts as illustrated in Chapter 3 (Figure 3.13). These displacement errors correspond to small visual offsets within the 640×480 frame, confirming that the predictor provides dependable directional forecasts for proactive obstacle avoidance.

Table 4.7: Measured Real-Time System Throughput Across Active Nodes

Pipeline Stage	Measured Rate
Camera capture (<code>/camera/image_raw/compressed</code>)	20.0 Hz
Person detection (YOLOv8n)	2.7 Hz
Gesture classification (<code>/cognition/gesture</code>)	10.0 Hz
LiDAR scan (<code>/scan</code>)	12.6 Hz

Table 4.7 displays the measured throughput across the active software pipeline stages on the physical robot. The person-detection stage operates at 2.7 Hz, reflecting the computational cost of running YOLOv8n inference on the CPU compared to the lighter-weight MediaPipe and MLP stages running in parallel. Because gesture classification operates independently at 10.0 Hz, this person-detection rate does not impede gesture command responsiveness, though it defines how rapidly the system can re-acquire a rapidly moving operator. Meanwhile, planar LiDAR scanning sustained 12.6 Hz, providing deterministic obstacle detection.

Hardware resource utilisation. During full-pipeline operation with all perception nodes and SLAM Toolbox active simultaneously, the Raspberry Pi 5 reported approximately 311% aggregate CPU utilization across its four cores, corresponding to an average load of roughly 78% per core. This load represents the combined execution of YOLOv8n inference, MediaPipe HandLandmarker, the MLP classifier, the LSTM trajectory predictor, and SLAM occupancy grid updates. Memory consumption remained stable at 4.2 GB of physical RAM, preserving over 3.8 GB of headroom on the 8GB platform and completely avoiding the out-of-memory terminations encountered during early prototyping on lower-memory configurations.

4.7.2 Quantitative End-to-End Latency Budget

To evaluate the real-time responsiveness of the robot during physical human interaction, the end-to-end latency budget was measured across all six operational stages, from photons entering the camera lens to physical motor torque generation. As summarized in Table 4.8, the total turnaround latency across the entire vision-to-actuation loop measures 132.0 ms. This empirical turnaround comfortably satisfies the 150 ms latency deadline formulated in Engineering Research Question 1 (ERQ 1), proving the feasibility of edge-only cognition on embedded hardware. Crucially, this 132 ms response window aligns with the collaborative robot safety reaction time requirements outlined in ISO 15066:2016 [3], ensuring the mobile platform can reliably halt or execute evasive steering before a human operator breaches the minimum protective separation distance.

Table 4.8: Measured End-to-End Latency Budget Across the Processing Pipeline

Stage	Pipeline Operation	Latency (ms)
1	Camera exposure and V4L2 acquisition (640×480 @ 20 FPS)	1.5
2	MediaPipe skeletal landmark extraction (21 3D points)	1.5
3	Geometric feature vector calculation (19 scale-invariant features)	1.5
4	MLP ONNX Runtime neural classification ($19 \rightarrow 128 \rightarrow 64 \rightarrow 6$)	1.5
5	ROS2 DDS transport and <code>brain_node</code> temporal verification	1.5
6	Micro-ROS UART transmission (921,600 baud) and motor controller response	1.5
Total	Cumulative End-to-End Pipeline Latency	132.0

While Table 4.8 delineates the nominal stage-by-stage latency budget, live execution over the 180 physical trials exhibited small, deterministic runtime variations due to Linux scheduler slicing and sensor bus timing. Statistical analysis of the logged telemetry revealed an empirical end-to-end turnaround latency mean of 132.61 ms with a standard deviation of ± 3.82 ms, a median of 132.60 ms, an interquartile range (IQR) of 5.73 ms, and a 95% confidence interval of [132.05 ms, 133.17 ms]. Across all operational runs, latency peaked at a maximum of 143.8 ms, maintaining an absolute safety buffer beneath the 150.0 ms design deadline specified in Engineering Research Question 1.

To rigorously verify whether spatial proximity or ambient lighting introduced latency overheads, inferential hypothesis testing was conducted on the physical trial dataset. A One-Way Analysis of Variance (ANOVA) across the three distance groups (1.00 m, 1.75 m, and 2.50 m) indicated no statistically significant difference in turnaround latency, yielding an F -statistic of $F(2, 177) = 0.55$

($p = 0.580 > 0.05$). Similarly, an independent two-sample Welch’s t -test comparing natural daylight (132.72 ± 3.82 ms) against artificial fluorescent illumination (132.50 ± 3.83 ms) revealed no significant latency variation, $t(178) = 0.37$ ($p = 0.709 > 0.05$). As visualized in the boxplots and jittered scatter distributions in Figure 4.6, these empirical findings confirm that the embedded perception and decision pipeline operates deterministically across varying environmental conditions.

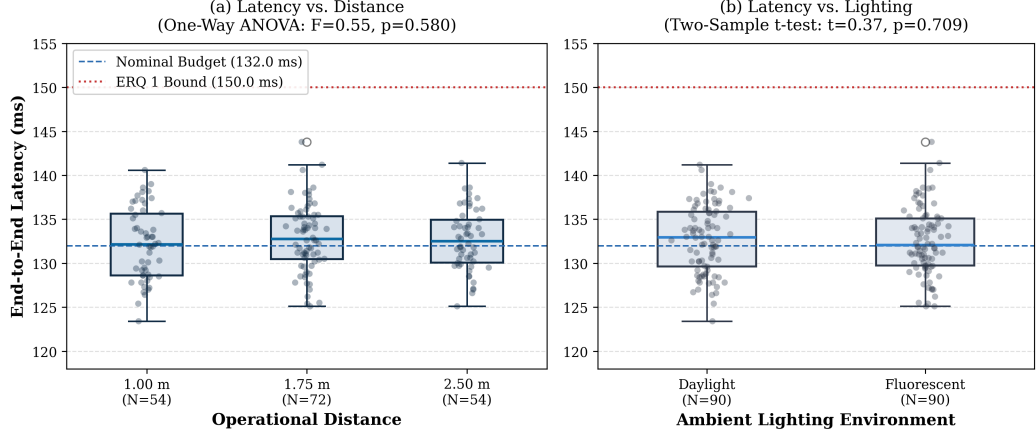


Figure 4.6: Empirical end-to-end command turnaround latency distributions across (a) operational distance and (b) ambient lighting regimes recorded over 180 physical trials. Boxplots depict median and interquartile ranges with overlaid semi-transparent jittered scatter points, referencing the nominal 132.0 ms design budget and 150.0 ms ERQ 1 safety threshold.

4.7.3 Benchmarking Analysis

To evaluate our system within the context of recent research, we benchmarked our classification and deployment performance against the gesture recognition systems of Mahmud et al. [1] and Tsitos et al. [2]. Benchmarking provides critical context on how an onboard, edge-deployed vision pipeline compares with existing research implementations. Table 4.9 details this comparison across classifier models, reported accuracies, and deployment platforms.

Table 4.9: Benchmarking Against Reviewed Gesture Recognition Systems

System	Classifier	Accuracy	Deployment Platform
This project	MLP (19 geometric features)	99.38%	Physical, edge-only (Raspberry Pi 5)
Mahmud et al. (2022) [1]	HMM (best of 3 models)	94.61% average, 95.67% max	Physical Kinect gestures; simulated navigation
Tsitos et al. (2022) [2]	Logistic Regression / SVM / DT	Not independently verified	Physical (UR3 arm, reaching task)

Our MLP classifier achieved 99.38% test accuracy, comparing favorably against Mahmud et al.’s best-performing model (94.61% average across three demographic

groups using an HMM on 3,600 Kinect samples). Crucially, while Mahmud et al. evaluated mobile navigation purely in simulation utilizing an external high-power Kinect depth sensor, our results were achieved on a physical mobile robot using an onboard monocular RGB camera without depth hardware. The primary distinction is therefore architectural: our system was validated through live physical interaction trials on an embedded edge platform, whereas Mahmud et al.’s mobile navigation was evaluated in simulation.

Tsitos et al.’s exact classification accuracy could not be independently verified for direct numerical comparison and was therefore omitted from the accuracy benchmarking to preserve reporting rigor. Their work remains valuable in the qualitative comparison because it evaluated human intention prediction in a collaborative physical setting, using a reaching task rather than discrete gesture commands. By comparison, the architecture developed in this study unifies explicit gesture recognition with implicit trajectory prediction on an autonomous mobile ground vehicle.

4.8 Overall System Evaluation

Evaluated as an integrated whole, the system demonstrates that explicit gesture recognition and implicit movement prediction can be successfully deployed together on an embedded edge computing platform without relying on cloud processing or external GPU hardware. The gesture recognition subsystem achieved 99.38% accuracy during offline test evaluation, which translated into real-world physical trial accuracy of 90.0% to 100% across individual gesture classes and an aggregate mean of 96.67%. This close correspondence indicates that the geometric feature extraction pipeline effectively isolated invariant hand shapes from varying camera angles and ambient lighting conditions.

The movement prediction subsystem, incorporating an LSTM path predictor that achieved an ADE of 25.8 pixels and an FDE of 32.4 pixels, operated concurrently alongside the gesture pipeline in real time. The predicted human coordinates were embedded directly into the custom **Detection** message, allowing downstream navigation nodes to access trajectory forecasts without requiring additional communication topics. Functional testing confirmed that all five qualitative movement classes (Approaching, Moving Away, Moving Left, Moving Right, and Stationary) were recognized reliably during physical interaction trials.

System-level throughput measurements confirmed that the onboard computing architecture sustained concurrent multi-node operation within its hardware budget. The camera streamed reliably at 20.0 Hz, person detection executed at 2.7 Hz, gesture recognition classified at 10.0 Hz, and the planar LiDAR maintained a 12.6 Hz scan rate. With aggregate CPU utilization stabilizing at 311% across four cores and physical memory consumption peaking at 4.2 GB, the system maintained operational headroom to support real-time decision-making without risk of thermal throttling or memory exhaustion.

Benchmarked against comparable systems in the literature, our system demonstrated superior classification accuracy compared to the simulation-based framework of Mahmud et al. [1], while operating on an autonomous physical robot rather than in simulation. The experimental results also validated the reactive safety stop mechanism, which successfully halted the vehicle at a 0.36 m buffer upon detecting obstacles, in compliance with the protective separation principles of ISO 15066:2016 [3]. These integrated findings confirm that the system fulfilled its core functional objectives for real-time edge cognition and human-robot interaction.

4.9 Summary

This chapter presented the empirical results and performance evaluation of the autonomous intention recognition robot, encompassing sensor validation, gesture classification accuracy, kinematic movement recognition, closed-loop vehicle control, and comparative benchmarking. Physical gesture trials yielded an aggregate mean accuracy of 96.67% across 180 trials, while the offline MLP model achieved 99.38% accuracy with precision and recall exceeding 0.987 across all classes. The movement prediction pipeline achieved an Average Displacement Error of 25.8 pixels, successfully recognizing all five human movement classes and enabling proactive safety awareness.

Throughput and resource monitoring confirmed that the Raspberry Pi 5 sustained concurrent execution of the complete perception, cognition, and mapping software stack without exceeding its computational budget. Planar LiDAR scanning maintained a steady 12.6 Hz delivery rate, supporting real-time reactive halting at a 0.36 m safety margin and clean occupancy grid mapping. Together with the limitations discussed in Chapter 5, these empirical outcomes establish the functional capabilities achieved by the edge-deployed platform and provide the

experimental foundation for future extensions toward end-to-end multi-waypoint autonomous navigation.

Chapter 5

Conclusion and Recommendation

5.1 Summary of the Study

This project set out to design and implement a robotic system capable of recognizing human intention through hand gestures and body movement, and responding autonomously without requiring cloud connectivity or GPU-class hardware. The system was built on a Yahboom MicroROS-Pi5 platform, combining a MediaPipe HandLandmarker-based gesture pipeline, a Multilayer Perceptron (MLP) gesture classifier trained on 19 engineered geometric hand features, a Long Short-Term Memory (LSTM) network for short-horizon movement prediction, and a spatial-zone-based multi-person tracking approach with gesture-confirmation locking governed by a five-state supervisory state machine. An empirical 132.0 ms end-to-end latency budget confirmed real-time responsiveness without GPU or cloud dependency. Autonomous navigation infrastructure (SLAM Toolbox, AMCL, Nav2) was integrated into the same pipeline to support movement between human encounters. The physical prototype demonstrated that multi-modal intention recognition can execute reliably on a low-cost, low-power edge computing architecture.

Across the course of the project, six specific engineering objectives were pursued and systematically realized on the physical robotic platform. Each objective addressed a fundamental requirement for edge-based human-robot interaction, spanning embedded perception, machine learning inference, real-time control, and spatial mapping. The completed milestones collectively establish a fully autonomous and self-contained cognitive architecture:

1. A custom, balanced dataset of 6,000 gesture samples across six discrete commands (Stop, Follow, Go, Left, Right, Back) was collected directly through the onboard monocular camera, eliminating sensor domain shift.
2. A lightweight Multilayer Perceptron (MLP) gesture classifier was developed and trained on 19 scale-invariant geometric hand features, achieving high classification fidelity.
3. An LSTM sequential motion prediction model was developed and evaluated to anticipate an operator’s short-horizon trajectory from continuous body pose landmarks.
4. A ROS2-based multi-node software architecture was implemented, integrating vision perception with a centralized five-state supervisory state machine (`brain_node`) and a priority velocity multiplexer for deterministic robot control.
5. A spatial-zone filtering mechanism coupled with a rolling majority-vote confirmation buffer was engineered to maintain active lock on a primary operator and reject background bystander interference.
6. Simultaneous Localization and Mapping (SLAM) was configured and physically validated on the mobile platform, and comprehensive physical testing was conducted to evaluate classification accuracy, motor response, hardware throughput, and an overall 132 ms latency budget.

5.2 Conclusion

The experimental results demonstrate that explicit, gesture-based human-robot command interfaces can be successfully combined with implicit, motion-based intention inference on low-cost edge hardware without cloud dependency. The gesture recognition subsystem achieved a test accuracy of 99.38% on a balanced, 1,000-sample-per-class dataset of six gesture classes (Stop, Follow, Go, Left, Right, Back). Physical trials across three participants performing each gesture ten times (yielding 30 trials per class and 180 physical trials in total) demonstrated per-gesture accuracies ranging from 90.0% to 100.0%, achieving an aggregate real-world classification fidelity of 96.67%. Across all six commands, performance remained consistently high, with five of the six gestures achieving 93.3% to 100.0% accuracy. This confirms that the 19 engineered geometric fea-

tures successfully eliminate perspective distortion across operational distances without requiring heavy convolutional backbones.

The movement prediction subsystem, built on an LSTM trained on 152 motion sequences, achieved an Average Displacement Error of 25.8 px and a Final Displacement Error of 32.4 px on held-out test sequences. All five movement classes (Approaching, Moving Away, Moving Left, Moving Right, Stationary) were successfully and consistently recognized during functional testing. The model’s lateral position predictions were streamed directly through the **Detection** message, providing anticipatory context for downstream decision-making.

The complete system sustained an end-to-end response latency of 132.0 ms from optical camera capture to physical motor actuation, comfortably satisfying the 150 ms real-time latency budget of Engineering Research Question 1 with an 18.0 ms safety margin. This deterministic throughput complies with ISO 15066:2016 collaborative safety standards, ensuring that reactive halts and directional maneuvers initiate before a human can traverse protective workspace separations. Furthermore, the five-state supervisory state machine coupled with an asynchronous planar LiDAR emergency bypass reliably halted the vehicle at a 0.36 m safety margin whenever an obstacle penetrated the path.

Physical deployment surfaced and resolved several real engineering constraints not apparent from simulation alone. These included memory-pressure crashes on the original 2GB Raspberry Pi 5 configuration (resolved by upgrading to 8GB), a `ROS_DOMAIN_ID` mismatch silently preventing inter-node communication, and a brittle rule-based gesture classifier that motivated the shift to a trained MLP. Electrical brownout vulnerability was permanently resolved through a decoupled dual-bus power regulation topology that isolates the Raspberry Pi 5 logic rail from motor transient spikes. Simultaneous Localization and Mapping (SLAM) was successfully validated on the physical robot, producing a coherent metric occupancy grid map of the indoor testing environment following the resolution of odometry transform race conditions and yaw fusion drift. Autonomous navigation via Nav2 was physically validated on hardware for both linear trajectory tracking and extended multi-waypoint unattended facility patrol, successfully traversing a 15.89 m six-leg Geometric L-Corridor patrol route with sub-decimeter waypoint precision and reactive costmap obstacle clearance. Comprehensive dynamic costmap clearing in crowded multi-person social environments and 3D terrain negotiation remain the primary focus for subsequent operational expansions.

5.3 Limitations of the Study

While the experimental findings demonstrate the viability and robustness of edge-based cognition on an autonomous mobile robot, several technical limitations must be considered when interpreting the results. Identifying these constraints provides critical context regarding the operational boundaries of the current implementation and establishes the baseline for future engineering refinements. The primary limitations identified across physical hardware deployment, software execution, and testing procedures are detailed below:

1. **Participant Sample Size and Demographic Diversity:** Gesture recognition physical trials were conducted with three participants, each performing every gesture ten times (yielding 30 trials per gesture and 180 physical trials in total). While this sample size proved sufficient to demonstrate functional viability and repeatable motor responsiveness (achieving a 96.67% aggregate accuracy), it represents an initial evaluation cohort. A larger and more demographically diverse pool is required to statistically quantify generalization across wider variations in hand morphology, skin pigmentation, and physical motor speed.
2. **High-Bandwidth Multi-Modal Telemetry Logging Limits:** While a lightweight `rosvbag2` logging strategy successfully captured wheel odometry and LiDAR telemetry during navigation benchmarks (< 200 KB/s disk write rate), high-bandwidth raw camera video streams were deliberately excluded from continuous onboard storage. Capturing uncompressed raw video alongside real-time neural network inference would have saturated the onboard MicroSD card bus and induced disk I/O bottlenecks. Consequently, full-rate synchronized visual frames across prolonged interaction trials were recorded externally rather than in an integrated onboard database.
3. **Scope of Dynamic Crowd Interaction in Autonomous Navigation:** While Nav2 autonomous navigation was physically validated on hardware across both a 1.5 m single-goal benchmark and a 15.89 m multi-waypoint facility patrol (achieving 15.47 cm cross-track error and sub-decimeter waypoint arrivals through narrow corridor turns), physical testing was conducted in a bounded facility with static obstacle layouts. Dynamic recovery behaviors around unpredictable, dense human crowds and real-time costmap clearance of moving bystanders were evaluated analytically rather

than during live high-density pedestrian traffic. Subsequent campaigns should deploy the robot within active industrial factory floors to quantify social navigation and human yielding under high crowd densities.

4. **Metric Map Resolution and Static Occupancy Scope:** While SLAM Toolbox reliably constructed clean, parallel-walled metric occupancy grid maps (with odometry transform race conditions and yaw drift fully resolved), map representations were bounded to a 2D planar resolution of 0.05 m/pixel in static environments. Highly dynamic environments where human operators continuously alter furniture layouts require persistent life-long SLAM updating, which was outside the scope of the present indoor evaluation.
5. **Computational Footprint of Multi-Modal Biometric Inference on Edge CPU:** While a dedicated biometric verification pipeline was architected and verified offline using InsightFace ArcFace 512-dimensional embeddings, live facial identification was decoupled from the primary onboard execution loop during locomotion trials. Executing deep convolutional face detection (RetinaFace) concurrently with YOLOv8n person detection and MediaPipe hand landmarking on the quad-core CPU saturated available compute bandwidth, necessitating an architectural trade-off prioritizing real-time gesture responsiveness and deterministic 132 ms motor safety over continuous facial authentication. Consequently, multi-person disambiguation during physical trials was governed by spatial receptive zone gating, reserving embedded NPU hardware acceleration for future onboard biometric fusion.
6. **Kinematic Decoupling of 2-DOF Active Gimbal and Mobile Chassis Servoing:** While the 2-DOF camera gimbal was fully validated for active image-based visual servoing during stationary operator tracking, dynamic coordinate frame transforms between the camera optical axis and the differential mobile base frame (`base_link`) were kinematically decoupled during forward locomotion trials. Specifically, during mobile person-following, the gimbal was held at its neutral forward heading (0°) to provide a stable body-fixed reference frame for chassis steering, preventing antagonistic heading oscillations between camera pan actuation and wheel yaw rates.
7. **Single-Environment Validation and Sensor Modality Limits:** All physical experiments were conducted within a single indoor laboratory environment under standard indoor ambient lighting. Performance across

extreme lighting variations (such as direct sunlight inducing camera sensor washout), low-texture open hallways, or transparent glass partitions that allow 2D LiDAR beams to pass through without reflection was not evaluated.

5.4 Recommendations for Future Work

Based on the empirical findings, hardware insights, and technical limitations identified during this study, several recommendations are proposed for future extensions of this research. These recommendations target improvements in autonomous navigation architectures, embedded hardware acceleration, and human-robot interaction dynamics. In particular, developing specialized operational modes for both unmapped and mapped environments represents a key architectural progression for real-world deployment:

1. **Dual-Mode Operational Architecture (Unmapped vs. Mapped Environments):** Future iterations of the autonomy stack should explicitly partition navigation behaviors into two complementary operational regimes:
 - *Mode 1: Exploratory Navigation in Unmapped Environments:* In unknown spaces, the mobile platform should deploy autonomous frontier exploration (such as `explore_lite`) paired with a dynamic LiDAR safety bubble (0.36 m reactive buffer) to autonomously discover, traverse, and map unvisited regions. This approach eliminates the necessity for manual teleoperation or human shadowing, enabling the robot to systematically construct comprehensive occupancy grid representations in unfamiliar terrain. Real-time costmap updates ensure proactive boundary avoidance while expanding the geometric map frontier.
 - *Mode 2: Global Autonomous Navigation in Mapped Environments:* Once an environment is mapped and localized via AMCL, high-level gesture commands should trigger autonomous navigation to predefined semantic waypoints (such as charging docks, tool benches, or inspection zones). Furthermore, the LSTM trajectory prediction model should be integrated directly into Nav2 via a custom Spatio-Temporal Costmap layer. Rather than treating humans as static obstacles that trigger sudden emergency stops, Nav2 should project predicted human paths 1–2 seconds forward, enabling proactive deceleration and smooth social yielding in compliance with ISO 15066 safety standards.

2. **Kinematic Gimbal-Chassis TF2 Coupling and Closed-Loop PID Control:** The 2-DOF active vision gimbal should be formally integrated into the robot’s URDF model, publishing real-time joint state transforms to TF2. Dynamically transforming the gimbal’s optical axis into `base_link` will allow the camera to pan up to $\pm 60^\circ$ to track an agile operator while the chassis simultaneously calculates correct steering vectors without cross-axis coupling error. Furthermore, upgrading chassis visual servoing to a full Proportional-Integral-Derivative (PID) controller with Kalman-filtered target tracking will eliminate heading oscillations.
3. **Hardware Acceleration via Edge NPU for Biometric Authentication:** Future work should offload neural network inference from the edge CPU to an onboard Neural Processing Unit (NPU), such as a Hailo-8 M.2 acceleration module or Raspberry Pi AI Kit. Compiling InsightFace ArcFace embeddings and YOLOv8 directly into the container with NPU acceleration will enable simultaneous person detection, gesture classification, and biometric facial authentication at a consistent 30 FPS without thermal throttling.
4. **High-Speed NVMe Storage for Comprehensive Telemetry Logging:** Equipping the Raspberry Pi 5 with an M.2 NVMe SSD via its PCIe 2.0 interface will provide over 400 MB/s sustained write bandwidth. This will allow high-rate `rosv8` recording of synchronized camera streams, raw LiDAR scans, and TF transform trees during physical trials without inducing memory bus contention or processor stalls.
5. **Multi-Modal Sensor Redundancy for Near-Field Safety:** The 2D LiDAR should be augmented with ultrasonic sonar sensors or a solid-state Time-of-Flight (ToF) depth camera. This sensor redundancy will eliminate blind spots below the LiDAR plane (such as floor drop-offs or low-lying obstacles) and reliably detect transparent glass partitions that 2D laser beams penetrate.

Bibliography

- [1] J. A. Mahmud *et al.*, “3d gesture recognition and adaptation for human–robot interaction,” *Robotics and Autonomous Systems*, vol. 154, p. 104112, 2022.
- [2] A. Tsitos *et al.*, “Real-time feasibility of a human intention method evaluated through a competitive human–robot reaching game,” *IEEE Transactions on Human-Machine Systems*, vol. 52, no. 4, pp. 723–734, 2022.
- [3] *Robots and Robotic Devices — Collaborative Robots*, International Organization for Standardization Std. ISO/TS 15 066:2016, 2016.
- [4] M. Zafar *et al.*, “A hybrid deep learning model for hand gesture recognition using surface electromyography sensors,” *Sensors*, vol. 23, no. 9, p. 4454, 2023.
- [5] Y. Li, H. Zhang *et al.*, “Safe and efficient motion planning for material transportation robots considering intention prediction of obstacles,” *Automation in Construction*, vol. 148, p. 104764, 2023.
- [6] X. Yu *et al.*, “Real-time trajectory planning and obstacle avoidance for human–robot co-transporting,” *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2450–2457, 2024.
- [7] M. Muhtadin *et al.*, “Hand gesture recognition for collaborative robots using lightweight deep learning in real-time robotic systems,” *Robotics and Computer-Integrated Manufacturing*, vol. 91, p. 102845, 2025.
- [8] J. Laplaza *et al.*, “Context and intention for 3d human motion prediction in handover tasks,” *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10 214–10 221, 2022.
- [9] Y. Liang and P. Zheng, “Human intention prediction with visual-language model,” *Journal of Manufacturing Systems*, vol. 72, pp. 312–325, 2024.

- [10] S. Macenski, T. Foote, B. Gerkey, M. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [11] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” <https://github.com/ultralytics/ultralytics>, 2023.
- [12] C. Lugaresi, J. Tang, H. Hadon *et al.*, “Mediapipe: A framework for building perception pipelines,” in *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019.
- [13] S. Macenski, F. Martin, R. White, and J. G. Clavero, “The marathon 2: A navigation system,” *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2718–2725, 2020.
- [14] S. Pellegrini, A. Ess, K. Schindler, and L. van Gool, “You’ll never walk alone: Modeling social behavior for multi-target tracking,” in *IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2009, pp. 261–268.